



Planeamento de Duty Cycles em Redes de Sensores

(Baseado em modelo de grafos e satisfação de restrições)

Supervisores: Diego Passos

Fernanda Passos

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Planeamento de Duty Cycles em Redes de Sensores

Usando Otimização com Restrições e Heurísticas

André Carrilho n.^a 49420 e-mail: a49420@alunos.isel.pt,

Supervisores:

Diego Passos

Fernanda Passos

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho 2026

Resumo

As redes de sensores sem fios constituem um dos pilares fundamentais da Internet das Coisas, sendo amplamente utilizadas em diversos contextos do mundo real, como a monitorização de queimadas florestais, a deteção de sismos, o controlo ambiental, a vigilância de infraestruturas críticas e a gestão térmica de centros de dados, onde as redes são compostas por um conjunto de sensores distribuídos, capazes de recolher e transmitir informação de forma autónoma através de ligações sem fios.

Na maioria destes cenários, os sensores operam com recursos muito limitados, nomeadamente ao nível da energia, uma vez que são frequentemente alimentados por baterias ou por fontes de energia difíceis de substituir. Com isto a eficiência energética assume um fator crítico para prolongar a vida útil da rede e garantir a sua operação contínua ao longo do tempo.

Uma abordagem comum para reduzir o consumo energético consiste na utilização de mecanismos de *duty cycling*, nos quais cada sensor alterna entre períodos de atividade e de inatividade. Ao reduzir o tempo em que o sensor mantém o seu rádio de comunicação ativo, é possível obter poupanças significativas de energia. No entanto, esta alternância impõe desafios adicionais, uma vez que os sensores passam a comunicar segundo escalonamentos temporais específicos.

Neste projeto é considerado um método específico de *duty cycle* designado por CPB (Co-Prime Based Schedule). Neste método, cada sensor opera segundo um escalonamento cíclico de ativação, que se repete ao longo do tempo, no qual o período de repetição desse escalonamento é um parâmetro fundamental do sistema. Para garantir que sensores vizinhos consigam comunicar mesmo em operação assíncrona, impõe-se que os períodos de repetição dos seus escalonamentos sejam **coprímos**, assegurando que, ao longo do tempo, existirão instantes em que sensores adjacentes estão simultaneamente ativos, possibilitando a comunicação.

A escolha destes parâmetros não é trivial, uma vez que deve respeitar simultaneamente restrições energéticas individuais e restrições de compatibilidade entre sensores vizinhos, configurando um problema de otimização combinatória.

O objetivo deste projeto consiste no desenvolvimento de um algoritmo capaz de calcular automaticamente parâmetros adequados para o método CPB, maximizando a compatibilidade da rede e minimizando o desvio relativamente aos *duty cycles* desejados. Adicionalmente, é desenvolvida uma aplicação com uma interface gráfica intuitiva que permite ao utilizador criar e editar topologias de redes de sensores, executar o algoritmo de planeamento e visualizar os resultados obtidos de forma clara e interativa.

Utilização de Inteligência Artificial

No desenvolvimento deste trabalho foram utilizadas **ferramentas de Inteligência Artificial generativa** como apoio complementar ao processo de concepção e implementação do projeto

Em particular, a Inteligência Artificial foi utilizada para **melhorar o desenho e a usabilidade da interface gráfica (frontend)** da aplicação. Este apoio incidiu sobretudo na definição e refinamento de estilos visuais, nomeadamente através do desenvolvimento do ficheiro `index.css`, com o objetivo de evitar uma interface excessivamente simples ou com fundo branco uniforme, tentando promover uma apresentação visual mais clara para o utilizador. De forma semelhante, a IA foi usada como suporte na configuração do ficheiro `eslint.config`, auxiliando na pesquisa de boas práticas de organização e legibilidade do código.

Adicionalmente, foram utilizadas ferramentas de IA na **geração de imagens ilustrativas**, em particular para a criação da **Figura 2 – Diagrama Entidade-Relacionamento (ER)**, seguindo parâmetros e requisitos definidos explicitamente de acordo com o projecto, de forma a garantir a coerência com o modelo conceptual do sistema desenvolvido.

Por fim a Inteligência Artificial foi também empregue como ferramenta de apoio à **pesquisa técnica**, nomeadamente na procura de bibliotecas adequadas para inclusão no ficheiro `build.gradle`, bem como na exploração preliminar da possibilidade de utilização do algoritmo **A*(astar)** no contexto do projeto. Embora esta abordagem tenha sido analisada, concluiu-se que a sua utilização não seria a mais adequada para a busca dos valores esperados, tendo sido, por isso, descartada.

Índice

<i>Resumo.....</i>	<i>3</i>
<i>Utilização de Inteligência Artificial</i>	<i>4</i>
<i>Índice.....</i>	<i>5</i>
<i>1 Introdução e Motivação.....</i>	<i>9</i>
<i>1.2 Requisitos do Sistema</i>	<i>10</i>
<i>1.2.1 Requisitos Funcionais</i>	<i>10</i>
<i>1.2.2 Requisitos Não Funcionais</i>	<i>11</i>
<i>1.3 Organização do Documento</i>	<i>11</i>
<i>2 Embasamento Teórico</i>	<i>12</i>
<i>2.1 Redes de Sensores Sem Fios.....</i>	<i>12</i>
<i>2.2 Duty Cycling.....</i>	<i>12</i>
<i>2.3 Duty Cycling Assíncrono Baseado em Escalonamento</i>	<i>13</i>
<i>2.4 Método CPB (Co-Prime Based Schedule)</i>	<i>13</i>
<i>3 Arquitetura.....</i>	<i>15</i>
<i>3.1Visão geral</i>	<i>15</i>
<i>3.2Tecnologias</i>	<i>16</i>
<i>3.3Diagrama Entidade-Relacionamento</i>	<i>17</i>
<i>4 Backend.....</i>	<i>19</i>
<i>4.1 JSON Objects</i>	<i>20</i>
<i>4.1.1 Formatos JSON de Entrada.....</i>	<i>20</i>
<i>4.1.2 Formatos JSON de Saída</i>	<i>22</i>
<i>4.3 Serviços</i>	<i>23</i>
<i>4.4 Camada de Repositório.....</i>	<i>23</i>
<i>4.5 Algoritmo Principal de Planeamento de Duty Cycles</i>	<i>24</i>
<i>4.5.1 Escolha do Sensor Inicial e Paralelização</i>	<i>24</i>
<i>4.5.2 Construção dos Domínios e Modelo de Restrições.....</i>	<i>25</i>
<i>4.5.3 Visão Geral do Algoritmo.....</i>	<i>26</i>
<i>4.5.3.1 Estrutura geral do algoritmo buildTree.....</i>	<i>28</i>
<i>4.5.3.2 Avaliação de Impacto e Propagação de Restrições(impactOf)</i>	<i>31</i>

4.5.3.3	<i>Heurísticas de Seleção de Variáveis</i>	34
4.5.3.4	<i>Ordenação de Valores e Exploração do Domínio</i>	35
4.5.3.5	<i>Aprendizagem de Conflitos (Nogoods)</i>	35
4.5.3.5.1	<i>Adição de nogoods globais — GlobalNogoodStore.add</i>	36
4.5.3.5.2	<i>Adição de nogoods locais — SearchContext.addLocalNogood</i>	36
4.5.3.5.3	<i>Verificação de conflitos — violatesNogood</i>	37
4.5.4	<i>Avaliação de Soluções</i>	37
4.5.5	<i>Resultado Final</i>	38
4.6	<i>Camada de Domínio</i>	38
4.6.1	<i>Entidade Sensor</i>	38
4.6.2	<i>Representação da Topologia da Rede</i>	38
4.6.3	<i>Topologia Planeada</i>	39
5.	<i>Frontend</i>	40
5.1	<i>Organização</i>	40
5.2	<i>Navigation</i>	40
5.3	<i>Pages</i>	42
5.3.1	<i>Dashboard (Visão Geral do Frontend / Menu Principal)</i>	42
5.3.2	<i>Página Networks (Gestão de Topologias)</i>	42
5.4	<i>Visão Geral dos Principais Componentes</i>	43
6.	<i>Testes de desempenho</i>	48
6.1	<i>Avaliação experimental — impacto do número máximo de vizinhos</i>	48
	<i>Análise dos resultados</i>	49
6.2	<i>Avaliação experimental — impacto do número de sensores (com paralelismo)</i>	49
	<i>Análise dos resultados</i>	50
6.3	<i>Avaliação experimental — impacto do número de sensores (sem paralelismo)</i>	50
	<i>Análise dos resultados</i>	51
6.4	<i>Comparação entre execução paralela e sequencial</i>	51
	<i>Análise dos resultados</i>	51
6.5	<i>Impacto do número de corrotinas na execução paralela</i>	52
	<i>Análise dos resultados</i>	52
7	<i>Conclusão</i>	54
8	<i>Apêndices</i>	55

Apêndice A.....	55
Apêndice B.....	57
Referências bibliográficas.....	58

Capítulo 1

Introdução e Motivação

As redes de sensores constituem um dos pilares fundamentais da Internet das Coisas (Internet of Things – IoT), permitindo a recolha e transmissão de informação em ambientes distribuídos e heterogéneos. Estas redes são frequentemente modeladas como grafos, nos quais cada nó representa um sensor e cada aresta representa uma relação de vizinhança entre sensores. Este modelo abstrato é amplamente utilizado na literatura por facilitar o estudo e o planeamento do comportamento global da rede [1].

Este tipo de rede surge em diversos contextos do mundo real, como a monitorização ambiental, a deteção de incêndios florestais, a vigilância sísmica e o controlo de condições ambientais em infraestruturas críticas, como centros de dados e instalações industriais [1]. Nestes cenários, os sensores encontram-se normalmente distribuídos por grandes áreas e operam de forma autónoma durante longos períodos de tempo.

Em muitas destas aplicações, os sensores apresentam fortes restrições ao nível da energia, do processamento e do armazenamento. Em particular, são frequentemente alimentados por baterias, o que torna inviável a sua operação contínua a longo prazo. Por este motivo, a eficiência energética assume um papel central no projeto e na operação de redes de sensores sem fios [1].

Uma técnica amplamente utilizada para reduzir o consumo energético é o *duty cycling*, no qual cada sensor alterna entre períodos de atividade e de inatividade, de acordo com uma determinada política. Durante os períodos de inatividade, os módulos de comunicação permanecem desligados, permitindo reduzir significativamente o consumo energético da rede [2]. Cada sensor possui, tipicamente, um *duty cycle* desejado, definido como a percentagem de tempo em que se encontra ativo, refletindo um compromisso entre consumo energético e capacidade de comunicação.

No contexto deste trabalho, considera-se uma família específica de mecanismos de *duty cycling*, designada por *schedule-based asynchronous duty cycling*. Neste tipo de abordagem, os sensores não recorrem a sincronização global, operando de forma assíncrona segundo escalonamentos cíclicos locais previamente definidos, onde cada sensor segue um padrão periódico de ativação e inatividade, repetido ao longo do tempo, sem depender de relógios globais ou mensagens de sincronização.

Neste modelo, é necessário garantir que sensores vizinhos possuem, ao longo do tempo, instantes em que os seus períodos ativos coincidem, de forma a permitir a comunicação direta entre eles. A determinação destes escalonamentos locais torna-se particularmente complexa em redes densas, uma vez que as decisões tomadas para um sensor influenciam diretamente as possibilidades de escalonamento dos seus vizinhos, originando dependências ao nível da topologia da rede.

Devido a esta complexidade, a definição manual de políticas de *duty cycling* torna-se impraticável à medida que o número de sensores aumenta. Surge, assim, a necessidade de uma solução automática que considere simultaneamente a topologia da rede e os requisitos individuais dos sensores. A aplicação desenvolvida neste projeto tem como objetivo receber uma descrição abstrata da rede — incluindo os sensores, os seus *duty cycles* desejados, as

respetivas tolerâncias (desvio máximo admissível relativamente ao valor desejado) e a topologia — e calcular automaticamente uma atribuição de ciclos de trabalho válida.

Esta aplicação pretende funcionar como uma ferramenta de apoio ao planeamento e à análise de redes de sensores, permitindo avaliar soluções viáveis de forma eficiente, sem depender da implementação física ou da experimentação direta em sensores reais.

1.2 Requisitos do Sistema

Nesta secção são apresentados os requisitos funcionais e não funcionais da aplicação desenvolvida neste projeto, tendo em consideração que estes requisitos foram definidos com base na proposta inicial e refletem as funcionalidades obrigatórias e opcionais do sistema.

1.2.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que a aplicação deve disponibilizar ao utilizador.

Requisitos Obrigatórios

- **Submissão de topologias de rede:**
A aplicação deve permitir ao utilizador submeter topologias de redes de sensores através de ficheiros no formato JSON (***JavaScript Object Notation***), incluindo os parâmetros para a geração automática de redes, como o número de sensores.
- **Parâmetros de entrada configuráveis:**
A aplicação deve permitir a configuração de parâmetros adicionais do planeamento, incluindo o *duty cycle* desejado para cada sensor e a tolerância máxima permitida. A tolerância tem como significado o desvio máximo aceitável entre o *duty cycle* pretendido e o *duty cycle* obtido após o planeamento pelo algoritmo.
- **Definição automática dos parâmetros de *duty cycle*:**
A aplicação deve utilizar um algoritmo de coloração de grafos aplicado à topologia da rede para atribuir automaticamente parâmetros de *duty cycle* aos sensores, tendo como responsabilidade garantir que sensores vizinhos tenham parâmetros com valores que respeitam as restrições implementadas no algoritmo.
- **Avaliação de desempenho computacional:**
O sistema deve recolher métricas de desempenho, nomeadamente o tempo de execução e a utilização de memória, permitindo analisar a escalabilidade da aplicação.
- **Interface web SPA:**
A aplicação deve disponibilizar uma interface web do tipo *Single Page Application* (SPA), acessível através de um navegador, permitindo a submissão de topologias e a visualização dos resultados do planeamento.
- **Desempenho:**
A aplicação deve garantir tempos de execução compatíveis com a utilização interativa no contexto de planeamento de redes de sensores, permitindo que o utilizador obtenha resultados em tempo útil mesmo para redes de maior dimensão. Desta

forma, assegura-se que o processo de planeamento é adequado ao uso prático da aplicação.

- **Exportação dos resultados:**

A aplicação deve permitir a exportação da topologia da rede juntamente com os resultados obtidos pelo algoritmo de planeamento num determinado formato (ex: svg).

Requisitos Opcionais

- **Base de dados de topologias e resultados:**

O sistema poderá permitir o armazenamento de topologias submetidas e dos respetivos resultados de planeamento, possibilitando futuras consultas ou comparações.

1.2.2 Requisitos Não Funcionais

Os requisitos não funcionais definem características globais da aplicação, relacionadas com qualidade, desempenho e usabilidade.

- **Escalabilidade:** O sistema deve suportar redes de sensores de diferentes dimensões, permitindo analisar o comportamento do algoritmo em cenários de maior escala. Contudo, a componente de visualização gráfica está circunscrita a um número restrito de sensores, devido às limitações físicas do *canvas* de desenho no *frontend*.

- **Testes:**

A aplicação deve incluir testes, com o objetivo de testar componentes críticos no *frontend*.

1.3 Organização do Documento

O presente relatório encontra-se organizado da seguinte forma.

No Capítulo 2 é descrita a arquitetura global da solução desenvolvida, incluindo a organização dos principais componentes do sistema, as tecnologias utilizadas e o modelo de dados adotado.

No Capítulo 3 é detalhada a implementação do *backend*, abrangendo os formatos de comunicação em JSON, a API REST, as diferentes camadas da aplicação e, em particular, o algoritmo de planeamento de *duty cycles* desenvolvido para resolver o problema.

No Capítulo 4 é apresentada a implementação do *frontend*, descrevendo a estrutura da aplicação web, as principais páginas e componentes, bem como os mecanismos de interação disponibilizados ao utilizador.

Por fim, o Capítulo 5 apresenta as conclusões do trabalho realizado, destacando os principais resultados obtidos e possíveis direções para trabalho futuro.

Capítulo 2

Embasamento Teórico

2.1 Redes de Sensores Sem Fios

As redes de sensores sem fios (*Wireless Sensor Networks* – WSN) são sistemas distribuídos constituídos por um conjunto de sensores capazes de observar o ambiente envolvente, processar localmente a informação recolhida e comunicá-la através de ligações sem fios. Cada sensor funciona como um nó da rede, colaborando com os restantes para permitir a recolha e encaminhamento de informação até um nó de destino ou até outro sistema responsável pelo seu processamento [1].

Do ponto de vista abstrato, uma rede de sensores pode ser representada por um grafo não dirigido, em que cada vértice representa um sensor e cada aresta representa uma ligação de comunicação entre dois sensores vizinhos. Esta representação é amplamente utilizada na literatura, uma vez que facilita a modelação da topologia da rede e a definição de algoritmos de comunicação e planeamento [1].

Por exemplo, considere-se a topologia apresentada na Figura 1.

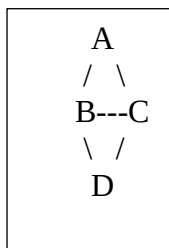


Figura 1-Exemplo simplificado de uma topologia

Nesta representação, o sensor **A** comunica diretamente com **B** e **C**, enquanto o sensor **D** comunica apenas com **B** e **C**. Qualquer comunicação entre **A** e **D** terá de ser realizada através de sensores intermédios.

2.2 Duty Cycling

Uma das principais limitações das redes de sensores é a restrita capacidade energética de cada dispositivo. Na maioria das aplicações, os sensores dependem de baterias cuja substituição é, frequentemente, difícil ou economicamente inviável. Como consequência, a minimização do consumo energético constitui um objetivo fundamental no planeamento destas redes [2].

A componente que mais energia consome num sensor é, normalmente, o **módulo de comunicação sem fios**. Por este motivo, uma técnica amplamente utilizada consiste em desligar periodicamente o rádio sempre que este não é necessário, sendo esta técnica conhecida como **duty cycling** [2].

Num mecanismo de *duty cycling*, cada sensor alterna entre dois estados:

- **Ativo (A)**, durante o qual o rádio permanece ligado e o sensor pode comunicar;
- **Inativo (I)**, durante o qual o rádio permanece desligado para reduzir o consumo energético.

Por exemplo, considere-se o seguinte escalonamento periódico:

Instante	1	2	3	4	5	6	7	8
Sensor	A	I	I	A	I	I	A	I

Neste exemplo, o sensor permanece ativo apenas durante três dos oito instantes apresentados. Durante os restantes instantes o rádio encontra-se desligado, reduzindo significativamente o consumo energético.

O **duty cycle** corresponde à percentagem de tempo em que o sensor permanece ativo relativamente ao tempo total de funcionamento, em que quanto menor for esse valor, menor será o consumo energético. Contudo, reduzir excessivamente o **duty cycle** também diminui as oportunidades de comunicação entre sensores, podendo degradar o desempenho da rede.

Existe, portanto, um compromisso entre eficiência energética e capacidade de comunicação, sendo necessário encontrar configurações que satisfaçam simultaneamente ambos os objetivos.

2.3 Duty Cycling Assíncrono Baseado em Escalonamento

Existem diversas famílias de mecanismos de *duty cycling*, que diferem na forma como coordenam os períodos de atividade dos sensores. Neste trabalho considera-se especificamente a família **Schedule-Based Asynchronous Duty Cycling** [2].

Nesta abordagem, os sensores não recorrem a qualquer sincronização global. Em vez disso, cada sensor segue autonomamente um escalonamento cíclico previamente definido, repetindo continuamente o mesmo padrão de atividade e inatividade.

Considere-se, por exemplo, dois sensores com os seguintes escalonamentos.

Instante	1	2	3	4	5	6
Sensor A	A	I	I	A	I	I
Sensor B	A	I	I	I	I	A

Cada sensor repete continuamente o respetivo escalonamento, independentemente do outro.

Como ambos funcionam de forma assíncrona, é necessário garantir que existirão instantes em que os dois sensores estarão simultaneamente ativos, permitindo a comunicação direta.

Caso isso nunca aconteça, os sensores deixam de conseguir trocar informação, comprometendo o funcionamento da rede.

Assim, o problema deixa de ser apenas minimizar o consumo energético, mas também garantir que os escalonamentos escolhidos permitem comunicação entre sensores vizinhos.

2.4 Método CPB (Co-Prime Based Schedule)

O método utilizado neste trabalho é o **CPB (Co-Prime Based Schedule)**, uma técnica pertencente à família dos métodos assíncronos baseados em escalonamento [3].

Neste método, cada sensor executa um escalonamento periódico cujo **período de repetição** deve ser coprimo relativamente ao período dos sensores vizinhos, em que por defeito, dois números são coprimos quando o seu máximo divisor comum é igual a 1.

Por exemplo:

- 5 e 8 são coprimos;
- 7 e 10 são coprimos;
- 6 e 9 não são coprimos, pois ambos são divisíveis por 3.

A utilização de períodos coprimos permite garantir, por propriedades da teoria dos números, que dois sensores vizinhos terão inevitavelmente instantes em que estarão simultaneamente ativos, mesmo sem qualquer sincronização global.

Considere-se, por exemplo, um sensor cujo escalonamento se repete de **5 em 5 instantes** e outro cujo escalonamento se repete de **7 em 7 instantes**.

Instante	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Sensor A	A	I	I	I	I	A	I	I	I	I	A	I	I	I	I
Sensor B	A	I	I	I	I	I	I	A	I	I	I	I	I	I	A

Embora os sensores operem de forma totalmente independente, existirão inevitavelmente instantes em que ambos estarão ativos simultaneamente.

No entanto, seleccionar estes períodos não é uma tarefa simples. Cada sensor possui um *duty cycle* desejado e uma tolerância admissível relativamente a esse valor. Assim, o período escolhido deve originar um *duty cycle* que respeite essa tolerância, minimizando simultaneamente o desvio relativamente ao *duty cycle* desejado.

Consequentemente, a determinação automática dos períodos CPB constitui um problema de otimização combinatória, sendo precisamente esse o problema abordado neste projeto.

Capítulo 3

Arquitetura

3.1 Visão geral

A aplicação desenvolvida segue uma arquitetura clássica cliente–servidor e encontra-se dividida em três componentes principais: **frontend**, **backend** e uma **base de dados**.

De forma geral, o **frontend** é responsável pela interação com o utilizador, permitindo a submissão de topologias e a visualização dos resultados, ao contrário do **backend** que concentra toda a lógica de processamento, incluindo a validação dos dados recebidos, a execução do algoritmo de planeamento baseado em coloração de grafos e a avaliação do desempenho computacional.

Em relação ao uso de **base de dados**, simplesmente permite o armazenamento persistente de topologias e resultados obtidos.

Por fim a comunicação entre o **frontend** e o **backend** é realizada através de uma API **REST** (Representational State Transfer), utilizando mensagens HTTP com dados em formato JSON.

Frontend (Aplicação Web SPA)

O **frontend** consiste numa aplicação web do tipo *Single Page Application* (SPA), acessível através de um navegador, tendo como principal objetivo fornecer uma interface gráfica simples e intuitiva para o utilizador, abstraindo a complexidade dos algoritmos subjacentes.

As principais responsabilidades do **frontend** são:

- Permitir a submissão de topologias da rede de sensores, quer através do envio de ficheiros JSON, quer através da configuração de parâmetros para geração de redes de acordo com o cliente;
- Possibilitar a configuração dos parâmetros de planeamento, como o número de sensores, o *duty cycle* desejado e a tolerância admissível;
- Apresentar os resultados do planeamento, incluindo os parâmetros de *duty cycle* atribuídos a cada sensor e a informação relativa às janelas de comunicação entre sensores vizinhos;
- Comunicar com o **backend** exclusivamente através da API REST, mantendo uma separação clara entre interface e lógica de negócio.

Backend (Servidor de Aplicação)

O **backend** é responsável por todo o processamento da aplicação e pela execução do algoritmo central do projeto, recebendo os pedidos enviados pelo **frontend**, validando os dados fornecidos executando o planeamento automático dos **duty cycles** com base na topologia da rede.

As responsabilidades do **backend** incluem:

- Validação das topologias submetidas pelo utilizador, garantindo a integridade e validade dos dados fornecidos;
- Execução do algoritmo de planeamento baseado em coloração de grafos;
- Determinação automática dos parâmetros do *duty cycle* de cada sensor, assegurando que sensores vizinhos recebem parâmetros adequados;
- Recolha de métricas de desempenho computacional, como tempo de execução e utilização de memória;
- Disponibilização dos resultados ao *frontend* através de *endpoints* REST.

A lógica da aplicação encontra-se organizada por camadas, separando o tratamento de pedidos HTTP (*Hypertext Transfer Protocol*), a lógica de negócio e, quando aplicável, o acesso à base de dados.

Base de Dados

A base de dados tem como finalidade armazenar topologias submetidas, configurações de planeamento e resultados obtidos, permitindo a reutilização de dados, a comparação entre diferentes execuções e a análise de resultados ao longo do tempo.

3.2Tecnologias

Nesta secção são apresentadas as principais tecnologias utilizadas no desenvolvimento da aplicação, organizadas de acordo com os diferentes componentes do sistema.

3.2.1 Frontend

O **frontend** da aplicação foi desenvolvido com tecnologias familiares para aplicações web, privilegiando a modularidade e a experiência do utilizador.

- **React** [4] foi utilizado para a construção da interface gráfica, devido ao seu modelo baseado em componentes reutilizáveis, realizado juntamente com o uso de **TypeScript**[5] permite maior segurança e deteção precoce de erros através de verificação de tipos;
- **Vite** [6] foi adotado como ferramenta de desenvolvimento, permitindo tempos de compilação reduzidos;
- **HTML5 Canvas API** [7]: utilizada para a renderização gráfica da rede de sensores, permitindo desenhar os nós (sensores) e as arestas (comunicações) de forma dinâmica e com elevado desempenho;
- **HTML, CSS e JavaScript/TypeScript** [8] foram usados para a estruturação, estilização e lógica da interface;

3.2.2 Backend

O **backend** foi desenvolvido com foco na robustez, clareza da lógica e facilidade de integração com o **frontend**.

- **Kotlin** [9] foi escolhido como linguagem principal, devido à sua expressividade e integração com o ecossistema Java;
- **Spring MVC / Spring Boot** [10] foi utilizado para a criação da aplicação servidor, facilitando a definição de controladores REST e a organização por camadas;
- A comunicação é realizada através de uma **API REST** [11], utilizando JSON como formato de troca de dados;
- **PostgreSQL** [12]: usada para gestão e armazenamento de dados relacionadas com o desenvolvimento da topologia criada pelo utilizador;

3.3 Diagrama Entidade-Relacionamento

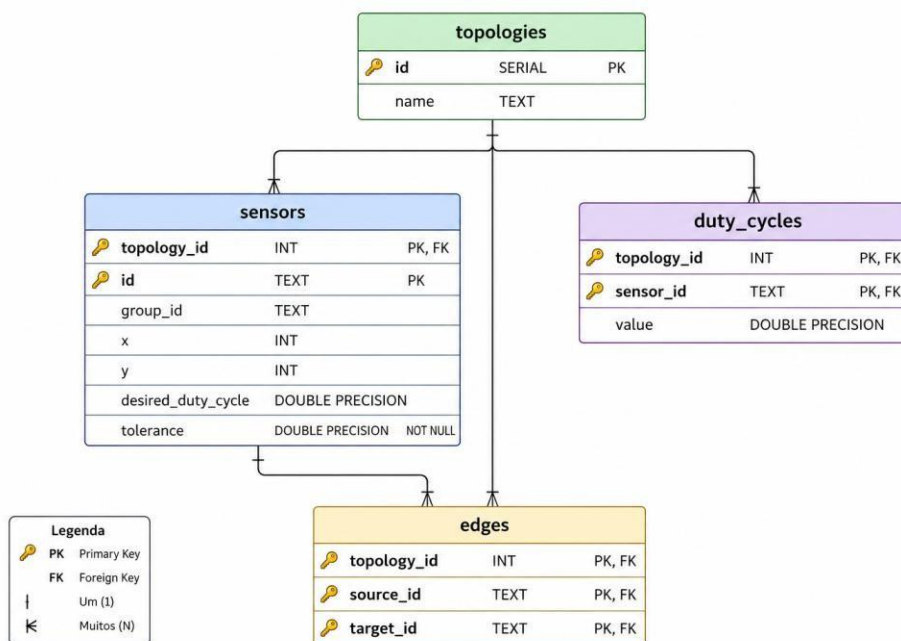


Figura 2 - Diagrama entidade-relacionamento (ER)

O Diagrama ER mostrado na Figura 1 representa a estrutura da base de dados, ilustrando as principais entidades e as relações entre elas que suportam o funcionamento da aplicação, em torno da tabela **topologia**, que agrega sensores, as ligações entre estes e os respectivos parâmetros operacionais.

Como observado na imagem, o diagrama é dividido em quatro entidades principais: **topologies**, **sensors**, **edges** e **duty_cycles**.

• **topologies**

A entidade **Topologies** representa uma configuração de rede armazenada no sistema. Inclui o atributo **id**, que constitui a chave primária e identifica univocamente cada topologia, e o atributo **name**, que armazena um identificador textual da topologia.

Cada topologia funciona como um contentor para um conjunto de sensores e para as relações de conectividade existentes entre eles.

• *sensors*

A entidade **Sensors** representa os nós individuais pertencentes a uma topologia. Cada sensor é identificado de forma única por uma **chave primária composta**, constituída pelos atributos `topology_id` e `id`, garantindo que os identificadores dos sensores são únicos apenas no contexto de uma determinada topologia.

Esta entidade inclui atributos como:

- `group_id`, que permite o agrupamento lógico de sensores;
- `x` e `y`, que representam a posição ou coordenadas do sensor no quadro de desenho;
- `desired_duty_cycle`, que define o valor alvo do ciclo de atividade;
- `tolerance`, que especifica a margem de desvio admissível relativamente ao valor pretendido.

Cada sensor pertence exatamente a uma topologia, estabelecendo-se assim uma relação de **muitos-para-um (N:1)** entre sensores e topologias.

• *edges*

A entidade **Edges** representa as ligações de comunicação ou conectividade entre sensores pertencentes à mesma topologia. Cada ligação é identificada por uma **chave primária composta** (`topology_id`, `source_id`, `target_id`), garantindo a unicidade de cada aresta dentro do contexto de uma topologia e de um par de sensores.

Os atributos `source_id` e `target_id` definem uma relação dirigida entre dois sensores, ambos referenciando a entidade **Sensors** através de uma chave estrangeira composta, garantido que apenas podem existir ligações entre sensores pertencentes à mesma topologia.

A entidade **Edges** também assegura a integridade referencial, usando políticas de remoção em cascata (*ON DELETE CASCADE*), o que implica que as ligações sejam automaticamente eliminadas quando um sensor ou uma topologia é removida.

• *duty_cycles*

A entidade **Duty_Cycles** armazena valores de execução ou valores calculados associados a cada sensor dentro de uma topologia. Cada registo é identificado por uma **chave primária composta** (`topology_id`, `sensor_id`), estabelecendo a ligação direta a um sensor específico.

O atributo `value` representa o valor do ciclo de atividade medido ou calculado para o sensor. Esta entidade mantém uma relação de **um-para-um** com a entidade **Sensors**, o que significa que cada sensor pode possuir, no máximo, um registo de ciclo de atividade por topologia.

Capítulo 4

Backend

O *backend* da aplicação segue uma arquitetura modular baseada em camadas, organizada de forma a separar responsabilidades entre domínio, serviços, acesso a dados e comunicação HTTP, facilitando a manutenção, escalabilidade e compreensão do sistema, permitindo que cada componente evolua de forma independente.

A Figura 2 apresenta a estrutura de pastas do *backend*, evidenciando a separação entre os principais módulos da aplicação.

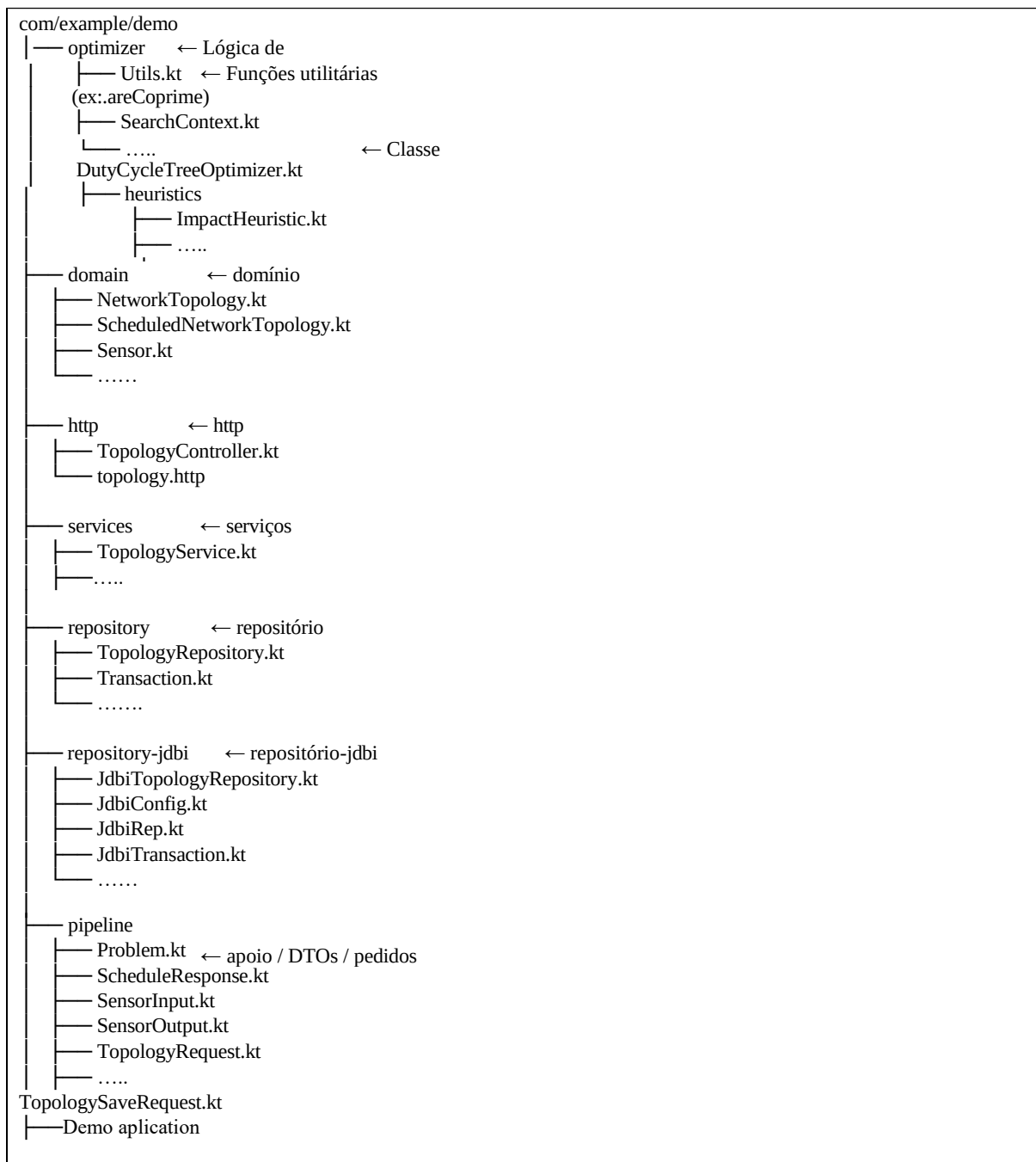


Figura 3– Estrutura modular do *backend*

- **domínio**: Módulo onde estão definidos todos os objetos utilizáveis em qualquer módulo do projeto.
- **optimizer** : Módulo responsável pela resolução do problema de otimização combinatória associado ao planeamento de *duty cycles* em redes de sensores. Implementa um algoritmo baseado em CSP (*Constraint Satisfaction Problem*), integrando técnicas de *backtracking*, heurísticas de seleção de variáveis (DSatur, MRV, grau do nó e risco de atribuição) e mecanismos de aprendizagem de *nogoods* que são esclarecidos nos capítulos seguintes.
- **http**: Módulo onde é realizado contacto inicial com um pedido HTTP, sendo enviado o contacto final com a resposta HTTP, funcionado juntamente com a camada serviços
- **serviços**: Módulo onde é implementada a lógica de negócio da aplicação, processando os pedidos vindos da camada HTTP, aplicando as regras de negócio coordenando a comunicação com a camada repositório (*repository*), garantindo também a gestão de transações e a execução de operações do sistema.
- **repositório**: Módulo onde são definidas as interfaces para acesso aos dados.
- **repositório-jdbi**: Módulo onde são implementadas interfaces utilizando JDBI, permitindo a ligação à base de dados.

4.1 JSON Objects

A comunicação entre o *frontend* e o *backend* da aplicação é realizada através de mensagens no formato **JSON**, transportadas sobre uma API REST. Estes objetos JSON definem contratos explícitos de entrada e saída, assegurando a interoperabilidade entre camadas, a validação dos dados recebidos e a independência entre o modelo de domínio interno e a representação externa dos dados.

Para este efeito, a aplicação distingue claramente entre **formatos de entrada (*Input Models*)** e **formatos de saída (*Output Models*)**, utilizados exclusivamente no contexto da comunicação HTTP, promovendo a separação de responsabilidades.

4.1.1 Formatos JSON de Entrada

Os formatos de entrada representam os dados enviados pelo cliente para o *backend*, quer para **persistência**, quer para **execução do algoritmo de planeamento**, sem armazenamento.

Criação e atualização de topologias

O formato de entrada para criação ou atualização de uma topologia inclui:

- Um identificador e nome da topologia;
- A lista de sensores, com os seus parâmetros (ex id,tolernace);
- A estrutura de conectividade entre sensores.

Estrutura geral do JSON de entrada para persistência:

```
{
  "name": "Topology A",
  "sensors": [
    {
      "id": "S1",
      "x": 10,
      "y": 20,
      "desiredDutyCycle": 50,
      "tolerance": 0.1,
      "groupId": "G1"
    },
    {
      "id": "S2",
      "x": 30,
      "y": 25,
      "desiredDutyCycle": 40,
      "tolerance": 0.5,
      "groupId": "G1"
    }
  ],
  "adjacency": {
    "S1": ["S2"],
    "S2": ["S1"]
  }
}
```

Os campos *x* e *y* representam coordenadas espaciais utilizadas pelo *frontend* para o posicionamento visual dos sensores, não influenciando diretamente o algoritmo de planeamento. O campo desiredDutyCycle representa o *duty cycle* desejado para o sensor, expresso em **percentagem** (50%,40%), enquanto tolerance indica a tolerância admissível relativamente a esse valor, utilizando a mesma representação ($40 \pm 0.5\%$, $50 \pm 0.1\%$). Por fim, o campo adjacency descreve a topologia da rede, especificando, para cada sensor, a lista dos sensores com os quais possui uma relação de vizinhança.

Planeamento sem persistência

Para a execução isolada do algoritmo de planeamento (sem armazenamento em base de dados), é utilizado um formato de entrada semelhante, mas sem informação identificativa da topologia.

Este formato permite testar e simular topologias, sendo particularmente útil durante o processo de desenho e validação no frontend.

Estrutura geral do JSON de entrada para calculo no algoritmo:

```
{
  "name": "Topology A",
  "sensors": [
    {
      "id": "S1",
      "x": 10,
      "y": 20,
      "desiredDutyCycle": 0.5,
      "tolerance": 0.1
    },
    {
      "id": "S2",
      "x": 30,
      "y": 25,
      "desiredDutyCycle": 0.4,
      "tolerance": 0.05
    }
  ]
}
```

```

"adjacency": {
  "S1": ["S2"],
  "S2": ["S1"]
}
}

```

4.1.2 Formatos JSON de Saída

Os formatos de saída representam as respostas devolvidas pela API após a execução de uma operação, incluindo resultados do algoritmo e métricas associadas.

Resposta de planeamento e simulação

O resultado do algoritmo de planeamento inclui:

- A lista de sensores com os parâmetros de duty cycle atribuídos;
- A topologia (estrutura de adjacência);
- Métricas de desempenho relativas à execução.

Estrutura geral do JSON de saída do cálculo do algoritmo:

```

"sensors": [
  {
    "id": "S1",
    "groupId": "G1",
    "x": 10,
    "y": 20,
    "dutyCycleParameter": 48->percentagem
  },
  {
    "id": "S2",
    "groupId": "G1",
    "x": 30,
    "y": 25,
    "dutyCycleParameter": 42->percentagem
  }
],
"adjacency": {
  "S1": ["S2"],
  "S2": ["S1"]
},
"performance": {
  "executionTimeMs": 12,
  "memoryUsedKb": 256 -> kilobytes
}

```

A Tabela 1 resume os principais *endpoints* da API da aplicação, especificando o método HTTP, o propósito de cada operação e os formatos de dados utilizados.

Tabela 1 – *Endpoints* da API REST para gestão e planeamento de topologias

Método HTTP	Endpoint	Descrição funcional	Entrada JSON	Saída JSON
POST	/api/topology	Criação de uma topologia	Sim	Sim
DELETE	/api/topology/{id}	Remoção de uma topologia armazenada	Não	Sim
PUT	/api/topology/{id}	Atualização e replaneamento de topologia	Sim	Sim
POST	/api/topology/planning	Execução do algoritmo de planeamento	Sim	Sim
GET	/api/topology/{id}	Consulta de topologia por identificador	Não	Sim

GET	/api/topology	Listagem de todas as topologias armazenadas	Não	Sim
-----	---------------	---	-----	-----

4.2 Camada HTTP (*Controller*)

A camada HTTP é responsável por estabelecer a comunicação entre o *frontend* e o *backend*, funcionando como ponto de entrada das requisições. Os pedidos HTTP são recebidos e encaminhados para a camada de *services*, onde é executada a lógica de negócio.

No projeto, esta camada é implementada pela classe `TopologyController`, que expõe um conjunto de *endpoints* REST associados à gestão de topologias onde cada método do controlador corresponde a uma operação específica do sistema, como criação, atualização, remoção, simulação e execução de planeamento.

4.3 Serviços

Na camada de *Services*, é implementada a lógica de negócio da aplicação, onde são processados os pedidos provenientes da camada HTTP, aplicando as regras de domínio e coordenando a interação com a camada de persistência (repositório) através do `TransactionManager`.

No caso da funcionalidade de gestão de *topologies*, a classe `TopologyService` é responsável por centralizar as operações principais do sistema, tais como criação, atualização, eliminação e execução do algoritmo das topologias.

Antes da execução das operações de persistência, os dados recebidos são convertidos para o modelo de domínio usado pelo repositório (`toDomain()`), garantindo o isolamento entre os modelos de entrada e a lógica interna da aplicação. Posteriormente, as operações relacionadas à base de dados são executadas dentro de transações, assegurando consistência dos dados.

Em funcionalidades como o “plan”, ao contrário dos outros é realizada a execução de algoritmos de planeamento, bem como a recolha de métricas de desempenho, nomeadamente o tempo de execução e consumo de memória.

Em caso de erros durante o processamento, são lançadas exceções específicas como `InvalidTopologyException`, `TopologyNotFoundException` ou `SchedulingFailedException`, permitindo um controlo do fluxo de erro.

4.4 Camada de Repositório

A camada de *Repository* é responsável pelo acesso direto à base de dados, sendo a única camada que contém operações SQL, ocorrendo a persistência e recuperação dos dados da aplicação, isolando completamente a lógica de negócio da implementação da base de dados.

No projeto, a implementação é realizada através do JDBI, permitindo a execução de *queries* SQL de forma simplificada e o mapeamento direto entre resultados da base de dados e objetos do domínio.

Estrutura de acesso a dados

O repositório segue uma abordagem baseada em SQL explícito, onde:

- Cada entidade do domínio é persistida em tabelas separadas (**topologies, sensors, edges, duty_cycles**)
- As relações entre entidades são reconstruídas manualmente durante a leitura da base de dados
- A consistência é garantida através da execução sequencial de operações SQL

4.5 Algoritmo Principal de Planeamento de Duty Cycles

Esta secção descreve o algoritmo desenvolvido para o planeamento automático dos *duty cycles* dos sensores, com base na topologia da rede e nos parâmetros definidos para cada sensor, partindo do modelo e das restrições previamente apresentados. O objetivo desta secção é explicar a estratégia adotada, as estruturas de dados utilizadas e as principais decisões de implementação.

O algoritmo proposto segue uma abordagem baseada em pesquisa em árvore, recorrendo a retrocesso (*backtracking*) para explorar o espaço de soluções possíveis.

4.5.1 Escolha do Sensor Inicial e Paralelização

A execução do algoritmo inicia-se através da seleção de sensores ordenados previamente com base em duas características principais:

- **o número de valores candidatos (domínio) que cada sensor pode assumir**, determinado a partir do *duty cycle* desejado e da respetiva tolerância;
- o número de vizinhos na topologia.

Esta ordenação inicial procura identificar sensores mais restritivos, ou seja, aqueles cuja atribuição antecipada tende a reduzir mais significativamente o espaço de pesquisa.

Adicionalmente, o algoritmo é executado de forma paralela. Cada execução é iniciada a partir de um sensor diferente, criando assim múltiplas árvores de pesquisa independentes, permitindo explorar diferentes regiões do espaço de soluções em simultâneo, tirando partido de arquiteturas *multi-core*.

Embora cada execução mantenha o seu estado local, existe partilha de informação global sob a forma de:

- melhor solução encontrada;
- conjunto global de conflitos (*nogoods* globais).

4.5.2 Construção dos Domínios e Modelo de Restrições

Antes do início da pesquisa, é construído o domínio inicial de cada sensor. Este corresponde ao conjunto de períodos admissíveis para o escalonamento do sensor, calculados a partir do *duty cycle* desejado e da respetiva tolerância.

$$T = \frac{100}{DC}$$

onde T representa o período e DC o *duty-cycle* expresso em percentagem.

Com base nesta relação, são definidos limites inferiores e superiores do intervalo de períodos admissíveis:

$$T_{min} = \frac{[100]}{DC+t}$$

$$T_{max} = \frac{[100]}{DC-t}$$

onde:

- DC é o *duty-cycle* desejado;
- t é a tolerância associada ao sensor.

Assim, o domínio de cada sensor é definido como o conjunto discreto de valores inteiros:

$$D(s) = \{T \in \mathbb{N} \mid T_{min} \leq T \leq T_{max}\}$$

Este pré-processamento reduz significativamente o espaço de pesquisa, eliminando desde o início valores inválidos.

As restrições do problema são essencialmente duas:

- o *duty cycle* atribuído a cada sensor deve pertencer ao domínio de valores admissíveis, determinado a partir do *duty cycle* desejado e da respetiva tolerância;
- os *duty cycles* atribuídos a sensores vizinhos devem respeitar as restrições de compatibilidade definidas pelo método CPB.

Para além destas restrições, o problema possui um objetivo, que consiste em minimizar o desvio entre o *duty cycle* atribuído a cada sensor e o respetivo *duty cycle* desejado. Esta minimização é efetuada através de uma função de custo baseada no erro quadrático médio (RMS), utilizada para comparar as soluções encontradas.

A **compatibilidade** entre sensores é definida através da propriedade de **coprimalidade** entre os respetivos períodos.

No contexto deste trabalho, esta propriedade (**coprimalidade**) é utilizada como restrição de compatibilidade entre sensores vizinhos, sendo imposto que sensores adjacentes devem selecionar períodos de *duty cycle* coprimos entre si.

4.5.3 Visão Geral do Algoritmo

O algoritmo principal encontra-se implementado no módulo de otimização, mais concretamente no ficheiro `DutyCycleTreeOptimizer.kt`. A função `buildTree()`, que constitui o núcleo do processo de pesquisa, sendo responsável pela expansão da árvore de busca, aplicação de retrocesso (*backtracking*) e avaliação das restrições do problema.

O algoritmo `buildTree` recebe como entrada:

- O conjunto de sensores a considerar;
- Para cada sensor, o *duty cycle* desejado e a respetiva tolerância.

A **saída** do algoritmo consiste numa atribuição de *duty cycles* aos sensores da topologia, podendo ser total ou parcial. A solução final não é retornada diretamente pela função `buildTree`, sendo armazenada implicitamente na variável global `bestAssignment`, atualizada dinamicamente ao longo da execução.

O processo de cálculo é incremental: os sensores são considerados um a um, sendo atribuídos valores compatíveis (coprimos entre vizinhos) com os sensores já processados. Sempre que uma escolha conduz a uma violação das restrições, o algoritmo recua e tenta alternativas diferentes. **A Figura 4 ilustra este processo através de um exemplo simplificado**, considerando quatro sensores A, B, C e D, com *duty cycles* desejados de 20%, 25%, 17% e 10%, respetivamente, e tolerância igual a 0%, ou seja, sem ser permitido qualquer desvio relativamente ao valor pretendido. Neste exemplo, apenas existem relações de vizinhança entre sensores consecutivos (A-B, B-C e C-D), sendo necessário garantir que sensores vizinhos recebem períodos **coprimos**.

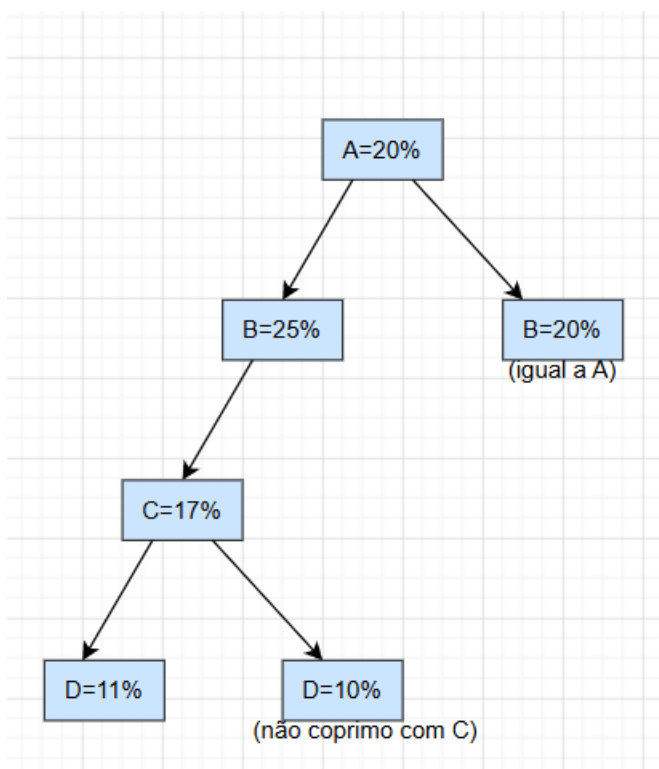


Figura 4 -Exemplo simplificado de busca

1. O algoritmo inicia a pesquisa atribuindo um *duty cycle* ao sensor **A**, uma vez que este foi selecionado pela heurística de ordenação inicial.
2. De seguida, tenta atribuir um valor ao sensor **B**, por este ser o próximo sensor escolhido pelas heurísticas de seleção, de seguida a atribuição **B = 20%** é rejeitada por violar a restrição de compatibilidade, uma vez que não satisfaz a condição de **coprimalidade** relativamente ao *duty cycle* que foi atribuído ao sensor **A**. O algoritmo então efetua retrocesso (*backtracking*) e testa a alternativa **B = 25%**, que é válida.
3. Após obter uma atribuição válida para **B**, o algoritmo prossegue para o sensor **C**, seguindo o mesmo processo de seleção e atribuição de um valor compatível(**coprímo**) com os sensores já processados (neste caso o vizinho B).
4. Para o sensor **D**, a primeira alternativa considerada (**D = 10%**) é rejeitada por violar novamente a restrição de compatibilidade com um dos sensores vizinhos já atribuídos. O algoritmo, então recua e experimenta a alternativa seguinte (**D = 11%**), que satisfaz todas as restrições.
5. Como todos os sensores possuem uma atribuição válida e compatível, o ramo é concluído e a solução é registada como candidata à melhor solução encontrada.

Definições do algoritmo

buildTree(sensor, domains, assignment, currentCost, countSensor, ctx)

- **assignment**: mapa parcial que associa sensores a valores de *duty-cycle* atribuídos, expressos em percentagem.
- **sensor**: sensor atualmente selecionado para atribuição de um valor de *duty-cycle*, representando o nó corrente na exploração da árvore de pesquisa. Este sensor é tratado como a variável ativa do problema de satisfação de restrições, sendo sobre o seu domínio que incide a tentativa de atribuição nesta invocação do algoritmo.
- **domains**: mapa que associa a cada sensor o conjunto de períodos ainda admissíveis para atribuição.
- **ctx** (*contexto de pesquisa*): estrutura auxiliar que armazena informação usada durante a exploração do espaço de soluções, nomeadamente:
 - **localNogoods**: conjunto de conflitos (*nogoods*) aprendidos localmente durante uma ramificação da pesquisa;
 - **globalNogoods**: conjunto global de conflitos partilhados entre diferentes explorações;
 - **bestAssignment**: melhor atribuição encontrada até ao momento;
 - **bestNullCount**: número mínimo de sensores sem atribuição na melhor solução;
 - **bestCost**: variável que armazena o valor do erro quadrático médio (RMS – *Root Mean Square*) da melhor solução encontrada até ao momento. O RMS é uma métrica que calcula a **diferença média entre os *duty cycles* atribuídos e os *duty cycles* desejados**, atribuindo maior peso a desvios de maior dimensão devido ao quadrado do erro, sendo utilizada para comparar soluções durante a pesquisa e atualizar a melhor solução sempre que é encontrada uma atribuição com menor erro.

Dois sensores adjacentes estão **em conflito** se:

- lhes for atribuído o mesmo valor de *duty-cycle*, ou os períodos correspondentes aos seus *duty-cycles* **não forem coprímicos**.

4.5.3.1 Estrutura geral do algoritmo *buildTree*

O algoritmo é detalhado nas secções seguintes por intermédio de blocos funcionais, permitindo uma análise isolada de cada etapa da sua lógica, no qual as referências às linhas apresentadas ao longo desta secção dizem respeito ao pseudocódigo do [Apêndice A](#)

Bloco 1 — Inicialização e verificação do domínio

(linhas 1–4)

Neste primeiro passo, o algoritmo obtém o domínio de valores possíveis para o sensor corrente em análise.

Caso o domínio esteja vazio, não existem valores possíveis para este sensor e a exploração deste ramo é interrompida.

```
1. domain ← domains[sensor]
2. Se domain = ∅ então
3.     retornar
4. Fim-se
```

Bloco 2 — Ordenação heurística dos valores do domínio

(linha 6)

Os valores do domínio são ordenados segundo critérios heurísticos, de forma a explorar primeiro as alternativas mais promissoras:

1. menor impacto esperado sobre sensores vizinhos (isto é, a escolha que elimina o menor número de valores possíveis dos domínios dos sensores vizinhos, preservando um maior número de alternativas para as etapas seguintes da pesquisa);
 2. menor desvio absoluto relativamente ao *duty cycle* desejado do sensor.
-

Bloco 3 — Tentativa de atribuição de valores

(linhas 7–9)

Para cada período admissível, é calculado o respetivo *duty cycle* em percentagem e efetuada uma tentativa de atribuição ao sensor.

```
7. para cada period ∈ domain fazer
8.     percentage ← [100 / period]
9.     assignment[sensor] ← percentage
```

Bloco 4 — Verificação de conflitos aprendidos (nogoods)

(linhas 10–13)

Antes de aprofundar a pesquisa, verifica-se se a atribuição corrente viola algum *nogood* (definido no ponto 4.5.7.3) previamente aprendido.

Em caso afirmativo, a atribuição é rejeitada de imediato.

```

10.     se assignment viola algum nogood em ctx então
11.         remover sensor de assignment
12.     Continuar *
13.     fim-se

```

* o termo “continuar” corresponde à instrução `continue` da linguagem Kotlin, que interrompe apenas a iteração corrente do ciclo sobre o domínio (`for (period ∈ domain)`), prosseguindo diretamente para o próximo valor candidato.

Bloco 5 — Cálculo do custo parcial da solução

(linhas 14–16)

Este bloco avalia a qualidade da atribuição parcial atual.

```

14.     erro ← percentage - sensor.desiredDutyCycle
15.     novoCusto ← currentCost + erro2
16.     rmsParcial ← √(novoCusto / |assignment|)

```

Interpretação formal:

- `erro` representa o desvio entre o *duty cycle* atribuído e o valor desejado pelo sensor;
- `novoCusto` corresponde à soma acumulada dos erros quadráticos;
- `rmsParcial` é o **erro quadrático médio (Root Mean Square)** da solução parcial, dado por:

$$RMs = \sqrt{\frac{1}{A} \sum_{D \in A} (dc_s - dc_s^{desired})^2}$$

* A é o conjunto de sensores com valores já atribuídos e dc corresponde ao *duty cycle*.

Bloco 6 — Poda baseada na melhor solução conhecida

(linhas 18–21)

Se já existir uma solução completa e a solução parcial atual apresentar um custo superior, o ramo é podado.

```

18.     se existe bestAssignment*
19.         e bestNullCount* = 0
20.         e rmsParcial > bestCost* então
21.             remover sensor de assignment
22.             continuar
23.     fim-se
24. *Variáveis globais

```

Bloco 7 — Verificação de conflitos locais com sensores vizinhos

(linhas 22–34)

Neste bloco é verificada a compatibilidade da atribuição atual com sensores adjacentes já atribuídos. Um conflito ocorre, como dito no ponto anterior (4.5.2) se:

- dois sensores adjacentes tiverem o mesmo *duty cycle*; ou
 - os períodos correspondentes não forem coprimos.
-

Bloco 8 — Aprendizagem de conflitos (nogoods)

(linhas 35–42)

Quando ocorre um conflito, é construído um *nogood*(4.5.7) contendo o conjunto mínimo de atribuições responsáveis pela violação, que é registrado para evitar explorações futuras equivalentes que levam ao mesmo conflito.

```

35.     se emConflito então
36.         conjuntoConflito ← subconjunto de assignment que viola
restrições
37.         conjuntoConflito ← conjuntoConflito  $\cup$  { sensor  $\rightarrow$  percentage }
38.         adicionar conjuntoConflito a ctx.localNogoods
39.         adicionar conjuntoConflito a ctx.globalNogoods
40.         remover sensor de assignment
41.         continuar
42.     fim-se

```

49. fim-para

Este mecanismo permite explorar sistematicamente o espaço de soluções, aprofundando cada ramo da árvore de pesquisa.

Bloco 12 — Retrocesso por impossibilidade de expansão do ramo

(linhas 50–54)

Este bloco é executado quando o sensor atual já não tem sensores vizinhos por explorar, mas a atribuição parcial ainda não inclui todos os sensores da topologia.

Nesta situação, não existem extensões possíveis para o ramo atual e a solução permanece incompleta. O algoritmo interrompe a exploração deste ramo e efetua **retrocesso**, regressando ao nível anterior da árvore de pesquisa para testar alternativas ainda não exploradas.

```
50.        senão se |assignment| < totalSensores então
51.
52.            remover sensor de assignment
53.            retornar
54.        fim-se
```

Bloco 14 — Retrocesso padrão de iteração

(linhas 55–56)

Após a exploração dos nós descendentes na árvore de pesquisa através das chamadas recursivas, o algoritmo executa o retrocesso (*backtracking*) para limpar o estado local antes de testar o próximo valor do domínio ou de retornar para o nível anterior da árvore.

```
55.
56. remover sensor de assignment
57. fim-para
```

4.5.3.2 Avaliação de Impacto e Propagação de Restrições(impactOf)

Durante a pesquisa, o algoritmo utiliza uma função de estimativa de impacto `impactOf`, aplicada na fase de **ordenação dos valores do domínio do sensor atual no início do procedimento `buildTree`**, antes da exploração do ciclo de atribuição dos períodos candidatos.

Esta função é usada como heurística de ordenação do domínio, estimando o efeito de cada possível atribuição sobre os sensores vizinhos ainda não atribuídos.

A função estima quantos valores seriam removidos dos domínios dos sensores adjacentes caso o sensor atualmente em análise recebesse um determinado valor. O impacto corresponde ao **retorno de um valor inteiro (Int)** que resulta da soma das reduções potenciais nos domínios dos sensores vizinhos ainda não atribuídos, funcionando como uma heurística do tipo *least constraining value*.

Este valor quantifica diretamente o grau de restrição introduzido por uma atribuição: quanto maior o impacto, maior a redução esperada no espaço de pesquisa dos vizinhos.

Deste modo, o algoritmo privilegia decisões que:

- preservam maior flexibilidade futura;
- reduzem o espaço de pesquisa dos sensores vizinhos de forma controlada.

Formalmente, quanto menor o impacto estimado, menor a perda de alternativas futuras, sendo essa atribuição considerada preferível durante a exploração da árvore de decisão.

Definições do algoritmo

impactOf(sensor, period, domains, assignment, ctx, topology)

- **sensor**: sensor atualmente em análise, para o qual se pretende avaliar o impacto de atribuir um determinado valor.
- **period**: período candidato (inteiro) pertencente ao domínio do sensor, correspondente a um *duty-cycle* em percentagem.
- **domains**: mapa que associa a cada sensor o conjunto de períodos ainda admissíveis.
- **assignment**: mapa parcial de atribuições já realizadas, associando sensores a valores de *duty-cycle* (em percentagem).
- **ctx (contexto de pesquisa)**: estrutura auxiliar que mantém:
 - `impactCache`: cache de impactos previamente calculados, indexada por (sensor, period, vizinho);
 - `coprimeCache`: cache de testes de com primalidade entre períodos.
- **topology**: representa a estrutura de ligação da rede de sensores sob a forma de grafo, sendo utilizada para obter a vizinhança de cada sensor através de `topology.neighbors(sensor)`.

4.5.3.2.1 Estrutura geral do algoritmo impactOf

O algoritmo (definido em [Apêndice B]) é descrito a seguir através de **blocos funcionais**, permitindo uma análise modular e rigorosa de cada etapa do processo de estimativa de impacto.

Bloco 1 — Conversão de Unidade e Inicialização

Linha(1-2)

Nesta etapa inicial, o algoritmo converte o período discreto em análise para o seu valor correspondente em percentagem de *duty-cycle*, inicializando a variável acumuladora do impacto total.

```
1: percentage ← [100 / period]
2: impactoTotal ← 0
```

Bloco 2 — Filtragem de Vizinhos Atribuídos e Consulta à Cache

Linha(3-11)

O algoritmo itera sobre os vizinhos do sensor. Se um vizinho já possui uma atribuição fixa no mapa `assignment`, ele é ignorado, pois o seu domínio já está reduzido a um único valor. Caso contrário, gera-se uma chave única (`chaveCache`) para verificar se o impacto deste período sobre este vizinho específico já foi calculado anteriormente nesta ramificação, reutilizando o valor em caso positivo.

```
3: Para cada neighbor ∈ topology.neighbors(sensor)      Faça
4:     Se neighbor ∈ assignment Então
5:         continuar *1
```



```

6:      Fim-se
7:      chaveCache ← (sensor, period, neighbor)
8:      Se ctx.impactCache contém chaveCache Então
9:          impactoTotal ← impactoTotal + ctx.impactCache[chaveCache]
10:         continuar *2
11:      Fim-se

```

***1**->indica que o vizinho atual já possui um valor fixo atribuído na solução parcial (assignment). Por isso, ele é ignorado na avaliação de impacto e o algoritmo salta imediatamente para o próximo vizinho do ciclo

***2** ->O algoritmo soma o valor recuperado ao impacto total (linha 9) e avança para a iteração do próximo vizinho

Bloco 3 — Avaliação de Compatibilidade Local e Contagem de Valores Válidos

Linha (12–19)

Quando o impacto não se encontra previamente calculado, o algoritmo analisa o domínio atual do sensor vizinho.

Cada valor presente nesse domínio é testado individualmente para verificar se, caso fosse convertido para percentagem, manteria a condição de coprimalidade (usando `areCoprimePercentages`) com o valor candidato do sensor atual.

Apenas os valores que respeitam esta condição são considerados válidos, sendo contabilizados como parte do domínio ainda preservado.

```

12:      domain ← domains[neighbor]
13:      Se domain = ∅ Então continuar *1
14:      valoresValidos ← 0
15:      Para cada valorVizinho ∈ domain Faça
16:          Se areCoprimePercentages(
              percentage,
              [100 / valorVizinho],
              ctx
          ) Então
17:              valoresValidos ← valoresValidos + 1
18:      Fim-se
19:      Fim-para

```

***1**-> Se o domínio do vizinho estiver vazio (ou seja, não houver períodos disponíveis para esse sensor nesta ramificação da procura), não há impacto a calcular nem valores a validar. O algoritmo interrompe a análise deste vizinho e passa para o próximo

Bloco 4 — Cálculo do Delta e Atualização da Cache

Linha (20–23)

O impacto individual deste par de sensores (*delta*) é determinado pela diferença entre o tamanho original do domínio do vizinho e a quantidade de valores que permaneceriam válidos. Este valor representa quantos períodos seriam eliminados do vizinho se esta decisão avançasse, sendo armazenado no `impactCache` e somado ao total acumulado antes de passar ao próximo vizinho.

```

20:      delta ← |domain| - valoresValidos
21:      ctx.impactCache[chaveCache] ← delta
22:      impactoTotal ← impactoTotal + delta
23:      Fim-para
24:      Retornar impactoTotal

```

Esta operação tem como objetivo calcular quantos valores são removidos dos domínios dos sensores vizinhos caso o sensor atual receba um determinado valor, devolvendo a soma total dessas remoções (impacto total nos domínios dos vizinhos), preferindo decisões que **reduzam o espaço de pesquisa dos vizinhos**.

4.5.3.3 Heurísticas de Seleção de Variáveis

A escolha do próximo sensor a atribuir é um dos pontos mais críticos do **algoritmo (4.5.3.1)**, influenciando diretamente a profundidade da pesquisa e o número de retrocessos.

A seleção é feita com base numa combinação de heurísticas clássicas de CSP (*Constraint Satisfaction Problem*), amplamente utilizadas em algoritmos de pesquisa com retrocesso [13], adaptadas ao contexto do problema:

1. Grau de Saturação (DSatur)

O grau de saturação (DSatur) é uma heurística utilizada para ordenar a seleção de sensores durante a fase de atribuição de valores, sendo uma estratégia clássica aplicada em **problemas de satisfação de restrições (CSP)** [14]. Esta heurística prioriza sensores que apresentam menor flexibilidade de atribuição futura, ou seja, sensores cujos vizinhos já possuem um maior número de *duty cycles* distintos atribuídos. Nestas situações, o sensor terá de ser compatível com um maior número de valores já escolhidos, **o que reduz as possibilidades de lhe atribuir um novo *duty cycle* válido**. Por este motivo, **estes sensores são processados prioritariamente**, permitindo detetar conflitos mais cedo e reduzindo o número de ramos inválidos explorados, bem como a necessidade de operações de retrocesso (backtracking)

O grau de saturação de um sensor é definido como o número de valores de *duty-cycle* distintos já atribuídos aos seus vizinhos processados no estado atual da pesquisa. Este valor é recalculado dinamicamente ao longo do algoritmo e é utilizado como critério de ordenação dos sensores a processar.

O cálculo deste critério é apresentado no **Algoritmo 1 — Cálculo do grau de saturação (DSatur)** e é aplicado diretamente no **Bloco 10 do algoritmo principal (Secção 4.5.3.1, linhas 44–45)**, onde os sensores ainda não atribuídos são ordenados antes da expansão do ramo de pesquisa.

Algoritmo 1: saturation(topology, sensor, assignment) 1:
valoresDistintos ← conjunto vazio
2: Para cada neighbor em topology.neighbors(sensor)¹ Faça 3:
 Se neighbor ∈ assignment Então
4: adicionar assignment[neighbor] em valoresDistintos
5: Fim-se
6: Fim-para
7: Retorne tamanho(valoresDistintos)

¹1->" topology.neighbors(sensor)" representa os vizinhos de um sensor

2. Minimum Remaining Values (MRV)

A heurística **MRV (Minimum Remaining Values)** é utilizada como segundo critério de ordenação no **Bloco 10 do algoritmo principal (Secção 4.5.3.1, linhas 44–45)**, tendo como função selecionar sensores com menor número de valores possíveis no domínio, reduzindo a probabilidade de falhas tardias, onde poucas opções restantes podem levar a impasses

3. Grau do Nó

A heurística do **grau do nó** é aplicada como terceiro critério de ordenação no **Bloco 10 do algoritmo principal (Secção 4.5.3.1, linhas 44–45)**, priorizando os sensores com maior número de vizinhos na topologia, uma vez que estes exercem maior influência estrutural sobre o problema de atribuição.

4. Risco de Nulo (*nullRisk*)

O risco de nulo (*nullRisk*) é utilizado como último critério de ordenação no **Bloco 10 do algoritmo principal (Secção 4.5.3.1, linhas 44–45)**. Esta métrica estima a probabilidade de um sensor ficar sem atribuição válida durante a pesquisa, combinando o número de vizinhos já atribuídos com o grau do sensor na topologia (isto é, o número de vizinhos diretamente conectados), permitindo identificar sensores mais “críticos” na estrutura da topologia. Sensores com maior risco de ficarem sem soluções válidas são priorizados na ordem de processamento, reduzindo a probabilidade de falhas tardias e retrocessos no algoritmo.

O cálculo desta heurística é formalizado no **algoritmo2 — *nullRisk***, que define a forma como o risco é computado com base nos vizinhos já atribuídos e no grau total de conectividade.

Algoritmo 2: *nullRisk*(topology, sensor, assignment)
 1: vizinhosAtribuidos <- 0;
 2: Para Cada neighbor em *topology.neighbors(sensor)* Faça
 3: Se neighbor está em assignment Então
 4: vizinhosAtribuidos <- vizinhosAtribuidos + 1;
 5: Fim-se
 6: Fim-para
 7: grauTotal <- Tamanho(*topology.neighbors(sensor)*);
 8: Retorne (vizinhosAtribuidos * 2) + grauTotal;

A combinação destas heurísticas permite uma ordenação dinâmica dos sensores durante a execução, ajustando-se continuamente ao estado parcial da solução.

4.5.3.4 Ordenação de Valores e Exploração do Domínio

Para cada sensor selecionado, os valores do seu **domínio** não são testados de forma arbitrária. Existe uma ordenação interna baseada em dois critérios principais:

- impacto local da escolha (*impactOf*);
- proximidade ao valor desejado do duty cycle.

Assim, são preferidos inicialmente valores que:

- reduzem menos o espaço de pesquisa dos vizinhos;
- minimizam o desvio face ao duty cycle pretendido.

Com esta estratégia melhoramos significativamente a qualidade das soluções encontradas nas fases iniciais da pesquisa.

4.5.3.5 Aprendizagem de Conflitos (Nogoods)

Sempre que uma atribuição conduz a uma violação de restrições, é gerado um **nogood** — uma combinação parcial de sensores e respetivos valores de duty-cycle que se provou inviável e que, por isso, deve ser evitada em futuras decisões do processo de pesquisa. Este mecanismo melhora significativamente a eficiência do algoritmo através da reutilização de

conhecimento sobre estados inválidos previamente identificados, evitando a repetição de erros já detetados.

São considerados dois níveis de nogoods:

- **Nogoods locais**, válidos apenas durante a execução corrente de uma coroutine, armazenados num mapa **privado** (`localNogoodIndex`) que associa sensores às suas respectivas listas de nogoods, sem necessidade de sincronização;
- **Nogoods globais**, partilhados entre todas as execuções paralelas, armazenados num `GlobalNogoodStore`, recorrendo a estruturas concorrentes (`ConcurrentHashMap` e `CopyOnWriteArrayList`) e sincronização fina por sensor.

Estrutura de armazenamento — índice por sensor

Tanto o índice local como o global utilizam **indexação por sensor**: cada sensor mapeia para a lista de nogoods em que participa.

Esta decisão é fundamental para a eficiência da verificação, pois em vez de iterar sobre todos os nogoods conhecidos, a função `violatesNogood` consulta apenas os nogoods associados ao sensor que está a ser atribuído no momento, reduzindo drasticamente o custo de cada verificação.

4.5.3.5.1 Adição de nogoods globais — `GlobalNogoodStore.add`

A função `add` insere um novo nogood no índice global garantindo três propriedades essenciais:

1. **Indexação por sensor**
O nogood é registado sob cada sensor que participa nas suas atribuições, permitindo que seja rapidamente localizado durante a verificação de conflitos.
2. **Subsumption (remoção de redundância)**
São removidos os nogoods existentes que sejam **subsumidos pelo novo nogood**, isto é, nogoods mais específicos cujas atribuições estão totalmente contidas no novo.
Se o novo nogood for mais geral (possuir menos atribuições), os mais específicos tornam-se redundantes e são eliminados(ex.: se já existe o nogood {A=5, B=4} e é adicionado o nogood {A=5}, o primeiro é removido porque sempre que {A=5} for violado, {A=5, B=4} também o será — o mais geral subsume o mais específico).
3. **Inserção ordenada por generalidade**
Caso o novo nogood não seja subsumido por nenhum existente, é inserido na lista numa posição que mantém a ordenação crescente pelo número de atribuições.
Desta forma, os nogoods mais gerais (com menos atribuições) são verificados primeiro, aumentando a probabilidade de deteção precoce de conflitos.

A sincronização é realizada através de **locks por sensor**, permitindo inserções concorrentes em sensores distintos sem bloqueio global.

4.5.3.5.2 Adição de nogoods locais — `SearchContext.addLocalNogood`

A adição de nogoods locais segue exatamente a mesma lógica de **subsumption** e **inserção ordenada** utilizada no índice global, mas opera sobre um `MutableList` privado de cada corrotina independente. Como não existe concorrência dentro do contexto local, não é necessária qualquer sincronização, eliminando completamente o overhead de locks.

4.5.3.5.3 Verificação de conflitos — violatesNogood

A verificação de conflitos é executada antes de cada decisão de atribuição.

A função consulta apenas os nogoods indexados pelo **sensor atualmente em atribuição**, tanto no índice local como no global.

Para cada nogood candidato são aplicadas duas otimizações fundamentais:

1. **Filtro por cardinalidade (short-circuit)**

Se o nogood tiver mais atribuições do que o estado atual (`nogood.assignments.size > assignment.size`), é impossível que todas as suas atribuições estejam presentes. Nesse caso, o *nogood* é imediatamente descartado.

2. **Verificação de correspondência completa**

Caso o filtro de cardinalidade seja satisfeito, verifica-se se todas as atribuições do *nogood* estão presentes no estado atual com os mesmos valores. Se tal acontecer, o estado é considerado conflituoso e a ramificação é descartada.

A consulta ao índice global utiliza `getFor(sensor)`, que devolve uma *snapshot* da lista associada ao sensor, garantindo leituras consistentes mesmo na presença de escritas concorrentes.

Algoritmo 3: violatesNogood(sesndor,assignment, ctx)

```

1: tamanhoAtribuido ← assignment.size
2:
3: // Verificar nogoods locais indexados pelo sensor
4: Para cada nogood ∈ ctx.localNogoodIndex[sensor] faça

5:   Se nogood.assignments.size ≤ tamanhoAtribuido então

6:     Se nogood.assignments.all((s, v) → assignment[s] == v)
então

7:       Retorne Verdadeiro
8:     Fim-se
9:   Fim-se
10: Fim-para
11:
12: // Verificar nogoods globais indexados pelo sensor
13: Para cada nogood ∈ ctx.globalNogoods.getFor(sensor) faça
14:   Se nogood.assignments.size ≤ tamanhoAtrib então
15:     Se nogood.assignments.all((s, v) → assignment[s] ==
v) então
16:       Retorne Verdadeiro
17:     Fim-se
18:   Fim-se

```

4.5.4 Avaliação de Soluções

Como referido na secção 4.5.3.1 Bloco **10** do algoritmo buildTree, a atualização da melhor solução encontrada é realizada através da função auxiliar *tryUpdate*, no qual a qualidade de cada solução é avaliada com base em dois critérios principais:

1. **Número de sensores não atribuídos** (critério principal)

2. Erro quadrático médio (RMS) entre os valores atribuídos e os valores desejados (Para cada sensor, o erro corresponde à diferença entre o *duty cycle* atribuído e o valor desejado, sendo o erro global obtido através da raiz do erro quadrático médio, permitindo uma comparação consistente entre soluções)

A função de avaliação privilegia primeiro a cobertura máxima da rede e, em caso de empate, a minimização do erro global.

4.5.5 Resultado Final

O resultado do algoritmo consiste numa atribuição parcial ou completa de *duty cycles* aos sensores da rede.

Quando não é possível satisfazer todos os sensores, o algoritmo devolve a melhor solução parcial encontrada, maximizando o número de sensores atribuídos e minimizando o valor do erro global.

Esta solução é posteriormente utilizada para:

- visualização de métricas de desempenho;
- visualização dos valores de *duty cycle* escolhidos pelo algoritmo

4.6 Camada de Domínio

A camada de domínio é responsável por representar os conceitos fundamentais do problema de planeamento de *duty cycles*, servindo de base ao funcionamento do algoritmo descrito na secção anterior. Esta camada define de forma clara os sensores da rede, as relações de vizinhança entre eles e a estrutura que armazena o resultado final do planeamento.

4.6.1 Entidade Sensor

A entidade **Sensor** representa um nó individual da rede de sensores e corresponde à unidade básica sobre a qual o algoritmo de planeamento opera, caracterizado por um conjunto de atributos que descrevem tanto a sua identidade como os seus requisitos operacionais.

```
data class Sensor(
    val id: String,
    val groupid: String? = null,
    val x: Int? = null,
    val y: Int? = null,
    val desiredDutyCycle: Double,
    val tolerance: Double = 0.0
)
```

4.6.2 Representação da Topologia da Rede

A estrutura da rede de sensores é modelada através da classe **NetworkTopology**, que representa explicitamente as relações de vizinhança entre sensores.

```
class NetworkTopology(
    val adjacency: Map<Sensor, List<Sensor>>
) {
    fun neighbors(sensor: Sensor): List<Sensor> {
        return adjacency[sensor] ?: emptyList()
    }

    fun sensors(): Set<Sensor> {
        return adjacency.keys
    }
}
```

Esta classe utiliza um mapa de adjacências para representar a topologia, onde cada sensor está associado à lista dos seus sensores vizinhos, sendo adequada ao problema em estudo, uma vez que as restrições de compatibilidade são definidas exclusivamente entre sensores adjacentes.

O método **neighbors** permite ao algoritmo obter, de forma simples, o conjunto de sensores vizinhos de um determinado sensor, sendo amplamente utilizado durante a verificação de restrições e a detecção de conflitos. Por fim o método `sensors` disponibiliza o conjunto total de sensores da topologia, permitindo inicializar o processo de pesquisa e percorrer todas as variáveis do problema.

4.6.3 Topologia Planeada

Ao contrário da estrutura anteriormente apresentada, a classe `ScheduledNetworkTopology` representa uma topologia **já sujeita ao processo de planeamento**, funcionando como um **modelo interno de domínio** utilizado após a execução do algoritmo. Esta estrutura é usada tanto para persistência em base de dados como para suporte às camadas superiores da aplicação.

```
data class ScheduledNetworkTopology (
    val name: String? = null,
    val adjacency: Map<Sensor, List<Sensor>>,
    val desiredDutyCycle: Map<Sensor, Double?>
)
```

Esta classe é constituída por:

1. estrutura de adjacências da topologia original, contendo a informação do sensor e seus vizinhos
2. O atributo `dutyCycles` associa a cada sensor o *duty cycle* atribuído pelo cliente.
3. O atributo opcional `name` permite identificar a topologia planeada

Dado que esta estrutura utiliza diretamente instâncias da entidade `Sensor` como chave, a classe assume um papel de **modelo de domínio interno**, não sendo exposta diretamente através da API REST. Para comunicação com o *frontend*, são utilizados modelos de saída específicos, conforme descrito na Secção 4.1.2

Capítulo 5

Frontend

5.1 Organização

No que diz respeito à organização dos ficheiros, o *frontend* foi estruturado em pastas separadas, conforme mostrado na figura 3, sendo cada uma responsável por um conjunto específico de responsabilidades dentro da aplicação, contribuindo para a manutenibilidade, escalabilidade e legibilidade do código, ao garantir uma distinção clara entre lógica de domínio, páginas, layout e componentes reutilizáveis.

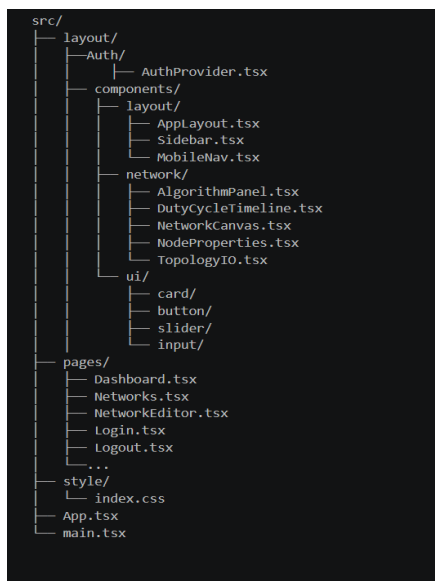


Figura 5 – Estrutura modular do frontend

- `layout/components/network/`: contém componentes específicos do domínio, responsáveis pela criação, edição, importação/exportação e execução de topologias de redes de sensores.
- `layout/components/layout/`: inclui componentes estruturais da interface, como o layout principal, a barra lateral (*sidebar*) e a navegação móvel.
- `layout/components/ui/`: agrupa componentes reutilizáveis de interface (botões, cartões, *inputs*, *sliders*), promovendo consistência visual e reutilização.
- `layout/Auth/`: inclui os componentes responsáveis pela autenticação e autorização da aplicação, efetuando a validação do estado de autenticação e o redireccionamento para as páginas de login quando necessário.
- `pages/`: define as principais vistas da aplicação (Dashboard, Networks, NetworkEditor).
- `style/`: contém a configuração global de estilos CSS.
- `App.tsx` / `main.tsx`: responsáveis pelo arranque da aplicação e configuração de rotas.

5.2 Navigation

Esta secção descreve a estrutura de navegação da aplicação web, ou seja, os diferentes caminhos (*routes*) que permitem ao utilizador deslocar-se entre as várias vistas do sistema. A

navegação é implementada utilizando a biblioteca **react-router-dom**, responsável pela gestão das rotas no *frontend*.

Ao contrário de outras aplicações que distinguem áreas públicas e privadas, a aplicação desenvolvida apresenta apenas uma secção principal. Todas as páginas estão acessíveis após o carregamento inicial da aplicação, não existindo, nesta fase, um sistema de autenticação associado à navegação.

A configuração das rotas é definida no componente principal da aplicação, garantindo uma organização simples e coerente entre as diferentes vistas.

Secção Principal da Aplicação

A aplicação é composta por uma única secção principal, que agrega todas as funcionalidades relacionadas com a visualização, gestão e edição de redes de sensores.

As rotas disponíveis são as seguintes:

- **/ (Dashboard):**
Página inicial da aplicação, apresentando uma visão geral do sistema, incluindo estatísticas globais sobre as redes de sensores, um resumo das topologias mais recentes e atalhos para a criação de novas redes ou edição de redes existentes.
- **/networks (Networks):**
Página dedicada à gestão das topologias existentes, permitindo visualizar todas as redes criadas, consultar informação básica sobre cada uma (como número de sensores e ligações), eliminar redes e aceder diretamente ao editor de uma topologia específica.
- **/editor/:id (Network Editor):**
Página principal de edição e interação com as redes de sensores. Nesta vista, o utilizador pode criar novas topologias ou editar topologias existentes, adicionar sensores, definir ligações entre nós e executar algoritmos de otimização.
O parâmetro `:id` identifica a topologia a editar. É utilizado um caso especial (`new`) para a criação de uma nova rede a partir do zero.

Exemplos:

- `/editor/new` – criação de uma nova topologia
- `/editor/12` – edição da topologia com identificador 12

Estrutura Global de Navegação

Todas as rotas da aplicação partilham uma estrutura comum, definida por um layout global. Este layout garante consistência visual entre as diferentes páginas, mantendo elementos comuns da interface, como menus e áreas de navegação, enquanto apenas o conteúdo principal é alterado.

Esta abordagem permite uma navegação fluida e coerente, facilitando a utilização da aplicação e contribuindo para uma melhor experiência do utilizador

5.3 Pages

5.3.1 Dashboard (Visão Geral do Frontend / Menu Principal)

O **Dashboard** (Figura 4) corresponde ao ponto de entrada principal da aplicação após o utilizador aceder ao sistema, tendo como função apresentar uma visão geral das topologias de redes de sensores existentes, permitindo ao utilizador obter rapidamente uma perceção do estado global do sistema e aceder às principais funcionalidades.

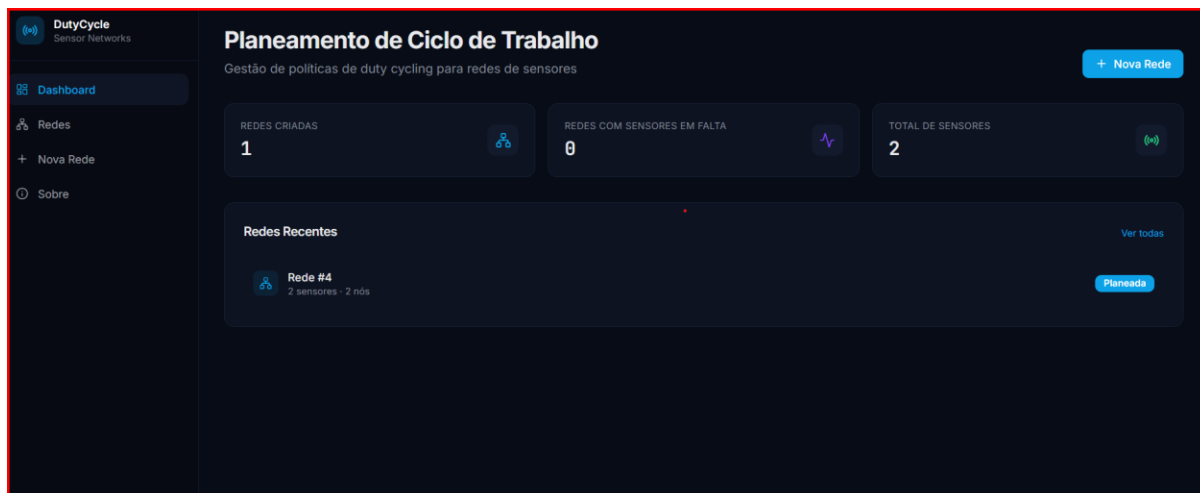


Figura 6-Representação do Dashboard

Tem como responsabilidade apresentar informação resumida, como o número total de redes criadas, o número total de sensores e o estado geral das topologias, podendo a partir do *Dashboard*, deixar o utilizador navegar diretamente para a criação de uma nova rede ou para a edição de redes já existentes.

É importante referir que o Dashboard não permite a modificação direta das topologias. O seu papel é essencialmente informativo e de encaminhamento para outras áreas da aplicação, nomeadamente o editor de redes. Os dados apresentados são obtidos a partir do *backend* quando o componente é carregado, no qual durante este processo, é gerido um estado de carregamento, garantindo que a interface responde de forma adequada enquanto os dados não estão disponíveis.

A interface do Dashboard encontra-se organizada em três partes principais:

- Uma secção inicial de apresentação da aplicação;
- Um conjunto de indicadores estatísticos globais;
- Uma lista das redes mais recentes, que permite acesso rápido ao editor.

5.3.2 Página Networks (Gestão de Topologias)

A página **Networks** (Figura 5) destina-se à gestão das topologias de redes de sensores existentes. Ao contrário do *Dashboard*, que apresenta apenas um resumo, esta página permite uma visualização mais detalhada e estruturada de todas as redes criadas no sistema, facilitando a organização e manutenção das topologias criadas.

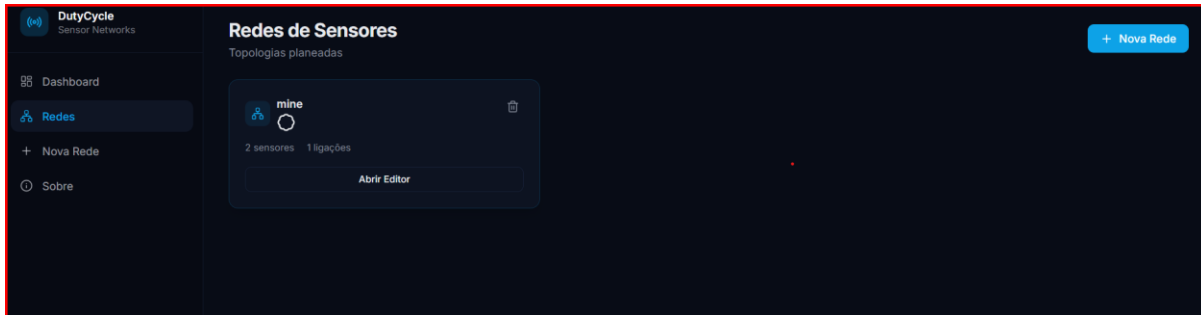


Figura 7- Representação de Networks

Nesta vista, o utilizador pode consultar a lista completa de topologias, aceder ao editor de uma rede específica ou remover redes que já não sejam necessárias, sendo o carregamento dos dados feito a partir do *backend*, apresentado um indicador visual enquanto a informação está a ser obtida.

A interface adapta-se a diferentes situações, apresentando:

- Um estado de carregamento, enquanto os dados são obtidos;
- Um estado vazio, caso não existam redes registadas;
- Uma lista de redes quando existem topologias disponíveis.

Cada rede apresentada contém informação, como o número de sensores e ligações existentes, bem como ações para edição ou remoção da topologia.

5.4 Visão Geral dos Principais Componentes

O **NetworkEditor**, responsável pela visualização e modificação de uma topologia, recorre a um conjunto de componentes do *frontend* que permitem a construção da topologia da rede e a execução do algoritmo de escalonamento no *backend*, sendo estes apresentados de seguida:

AlgorithmPanel

O componente **AlgorithmPanel** funciona como a interface de controlo para a execução do algoritmo de escalonamento no *backend*.

Este componente disponibiliza:

- Um botão principal para executar o algoritmo
- Uma opção de *reset* quando existem resultados disponíveis
- Um *slider* desativado, reservado para futura afinação de parâmetros
- Texto informativo que descreve as restrições do algoritmo

O **AlgorithmPanel** é intencionalmente leve, não executando qualquer tipo de computação local. A responsabilidade pela execução é delegada ao componente **NetworkEditor**, garantindo uma separação clara entre a interface de utilizador e a lógica de negócio.

DutyCycleTimeline

O componente **DutyCycleTimeline** é responsável por apresentar os resultados do algoritmo de escalonamento de forma estruturada e visual.

Apresenta:

- O *duty cycle* calculado para cada sensor
- Uma representação gráfica em barra da distribuição do *duty cycle*
- Métricas de execução (tempo e utilização de memória)
- Mensagens opcionais do *backend* relativas ao resultado da execução

Cada sensor é apresentado numa linha contendo o seu identificador, o valor calculado e uma barra proporcional, permitindo uma interpretação rápida e intuitiva dos resultados do algoritmo.

NetworkCanvas

O **NetworkCanvas** (Figura 6) é o principal componente visual do sistema, responsável pela renderização e interação com a topologia da rede de sensores, utilizando API HTML5 Canvas para renderizar de forma eficiente:

- Nós de sensores como elementos circulares
- Arestas como ligações entre sensores
- Atribuições de grupos e cores para agrupamento lógico

Suporta ainda múltiplos modos de interação:

- Criação de nós por duplo clique
- Reposicionamento por *drag-and-drop*
- Criação de ligações entre nós (definição dos vizinhos)
- Seleção e edição de nós
- *Feedback* visual para grupos e resultados do algoritmo

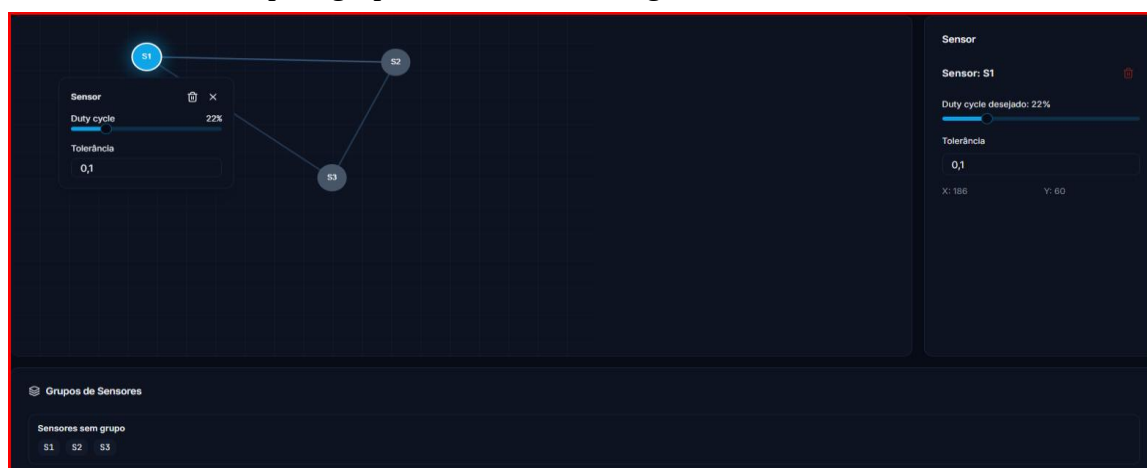


Figura 8->Demonstração do **NetworkCanvas**

Este componente atua como o editor espacial principal do sistema, traduzindo o modelo abstrato do grafo numa representação visual interativa.

NetworkEditor (Orquestrador do Frontend)

O componente **NetworkEditor** é o orquestrador central de toda a aplicação *frontend*. É responsável por gerir o ciclo de vida completo de uma topologia de rede de sensores, funcionando como a ponte entre a interação do utilizador, os componentes visuais e os serviços de *backend*.

Ao contrário dos componentes de interface individuais, o **NetworkEditor** não se foca numa única responsabilidade. Em vez disso, coordena a gestão de estado, as interações do utilizador, a persistência de dados e a execução do algoritmo.

Gestão de Estado e Modelo de Grafo

O editor mantém a topologia utilizando *React state hooks*, representando o sistema como um grafo composto por:

- `SensorNode[ ]` → nós espaciais com parâmetros de configuração
- `Edge[ ]` → ligações bidirecionais entre sensores
- `selectedNodeId` → nó atualmente ativo
- `selectedNodes` → seleção múltipla para agrupamento
- `edgeMode` e `groupMode` → estados de interação
- `result` → resultado da execução do algoritmo no *backend*

Cada nó de sensor contém:

- Posição espacial (x, y)
- *Duty cycle* desejado
- Valor de tolerância
- Identificador de grupo opcional

Esta estrutura permite representar simultaneamente a geometria espacial e a organização lógica dos sensores.

Construção e Sincronização do Grafo

A topologia é construída dinamicamente e sincronizada com os dados do *backend* através do React Query, quando se tenta modificar ou visualizar uma topologia existente, ou seja, quando uma topologia existente é carregada:

- Os dados dos sensores são transformados em objetos `SensorNode`
- As listas de adjacência são convertidas em *arrays* de arestas
- Arestas duplicadas são removidas através de um sistema de chaves normalizadas (`src | target`)
- Os identificadores de grupo são preservados, quando existentes

Este processo garante que o *frontend* mantém sempre uma representação consistente e canónica do modelo de grafo do *backend*.

Modelo de Interação do Utilizador

A interação do utilizador é tratada através de um conjunto de *callbacks* controlados:

- `handleAddNode` → criação de novos sensores no *canvas*
- `handleMoveNode` → atualização dinâmica da posição dos nós
- `handleSelectNode` → gestão de seleção, criação de ligações e lógica de agrupamento
- `handleUpdateNode` → atualização de propriedades dos nós ou propagação de alterações entre grupos
- `handleDeleteNode` → remoção de nós e das ligações associadas

Os modos de interação influenciam diretamente o comportamento do sistema:

- Em modo de arestas, os cliques estabelecem ligações entre nós
- Em modo de grupo, múltiplos nós podem ser selecionados e agrupados
- Em modo normal, a seleção foca-se na edição individual de nós

Mecanismo de Agrupamento

O sistema suporta o agrupamento lógico de sensores, permitindo a configuração em lote de parâmetros, facilitando a gestão eficiente de redes de sensores de grande dimensão.

Os grupos são gerados dinamicamente através de um sistema incremental de identificadores (por exemplo, G1, G2, ...). Após a criação de um grupo:

- Todos os nós selecionados recebem o mesmo `groupId`
- Alterações aplicadas a um nó propagam-se a todos os nós do mesmo grupo
- Os grupos são preservados visual e logicamente durante a persistência e execução do algoritmo

Persistência e Integração com o Backend

O editor comunica com o *backend* através de duas operações principais:

- `saveTopology` → salvamento manual da topologia presente na base de dados
- `runAlgorithm` → execução da lógica de otimização no *backend*

Antes do salvamento, o grafo é serializado em:

- Uma lista estruturada de sensores
- Um mapa de adjacência bidirecional

A utilização de uma arquitetura baseada em *mutations* com React Query assegura:

- Eliminação de pedidos duplicados
- Tratamento de erros através de notificações (*toast*)
- Consistência do estado da interface durante operações assíncronas

Pipeline de Execução do Algoritmo de Planeamento de Duty Cycle

Quando o utilizador executa o algoritmo:

1. O estado atual do grafo é convertido num `TopologyRequest`
2. As coordenadas dos sensores são normalizadas (valores arredondados)
3. A estrutura de adjacência é reconstruída
4. O pedido é enviado para o endpoint do backend
5. A resposta é armazenada em `result` com os valores calculados pelo algoritmo

Os resultados são posteriormente propagados para componentes de visualização, tais como:

- `DutyCycleTimeline`
- `NodeProperties` (quando atualizadas dinamicamente)
- Visualização de métricas de

desempenho Este processo estabelece um

pipeline reativo:

Editar → **Executar** → **Visualizar Resultados**

Composição da Interface

O **NetworkEditor** integra vários componentes numa interface unificada:

- `NetworkCanvas` → editor espacial do grafo, permitindo desenhar a topologia
- `NodeProperties` → painel de configuração de nós, permitindo visualizar os valores ou modificar os valores de *duty cycle*
- `AlgorithmPanel` → controlos de execução do algoritmo
- `DutyCycleTimeline` → visualização dos resultados desenvolvido pelo algoritmo
- `TopologyIO` → funcionalidades de importação/exportação (ex:

svg) A interface está organizada em:

- Área principal de interação com o *canvas*
- Painel lateral para propriedades dos nós
- Secção inferior para agrupamento e visualização de resultados
- Barra superior de controlo para ações globais

Esta arquitetura garante que o **NetworkEditor** funciona como a camada central de coordenação de todo o sistema *frontend*.

Capítulo 6

Testes de desempenho

Nesta secção analisa-se o impacto da utilização da heurística de aprendizagem de conflitos (*nogood learning*) e do paralelismo no tempo de execução do algoritmo, considerando diferentes topologias de redes de sensores.

Os testes foram realizados utilizando o **JMH (Java Microbenchmark Harness)**, com execução repetida em múltiplos forks, de forma a garantir maior robustez e fiabilidade dos resultados obtidos, em que cada fork corresponde a uma execução independente da benchmark numa nova instância da JVM, permitindo mitigar efeitos de aquecimento (*warm-up*), otimizações do compilador JIT (*Just-In-Time*) e interferências de execuções anteriores. Desta forma, os tempos medidos refletem de forma mais consistente o comportamento real do algoritmo.

Os testes experimentais foram executados num computador com um processador **AMD Ryzen 5 5600H** com **Radeon Graphics, 12 GB** de memória **RAM** e sistema operativo Windows 11 de **64 bits**, sabendo que este tipo de processador **possui 6 núcleos físicos (physical cores) e 12 núcleos lógicos**.

As topologias utilizadas nos testes foram geradas através de uma função pseudoaleatória, responsável por:

- definir o número de sensores da rede;
- impor um limite superior ao número máximo de vizinhos por sensor;
- fixar o valor da tolerância;
- gerar valores iniciais aleatórios de *duty cycle* para cada sensor.

6.1 Avaliação experimental — impacto do número máximo de vizinhos

Nesta secção analisa-se o impacto do **número máximo de vizinhos por sensor** no desempenho do algoritmo, comparando a execução **com aprendizagem de conflitos (*nogood learning*) e sem o uso do mesmo**. O parâmetro *máximo de vizinhos* controla a densidade da topologia gerada aleatoriamente e influencia diretamente o número de restrições e conflitos potenciais durante a pesquisa.

Metodologia experimental

Para cada valor do número máximo de vizinhos, foi gerada uma topologia aleatória com **30 sensores** e tolerância 0,5% para cada sensor, com a assistência da função de geração pseudoaleatória. A topologia resultante é posteriormente carregada e utilizada de forma idêntica em todas as execuções do *benchmark*, garantindo comparabilidade entre testes.

Os testes foram realizados com as seguintes configurações:

- **Número de forks:** 5
- **Cada fork executa uma instância completa do algoritmo**
- **Métrica principal:** tempo total de execução (em milissegundos)

Para cada configuração, o **tempo médio** apresentado corresponde à média aritmética dos tempos obtidos nos 5 *forks* (no qual realizei 2 vezes os testes por número de vizinhos, tendo 10 *forks* para cada teste).

Nº máximo de vizinhos	WITH NOGOOD — Tempo médio (ms)	WITHOUT NOGOOD — Tempo médio (ms)
3	432,4	432,3
5	4 391,7	5 966,8
7	74 574,8	106 437,7

Análise dos resultados

Os resultados mostram que, para topologias pouco densas (máximo de 3 vizinhos), o impacto da aprendizagem de conflitos é significativamente pequena, vendo que nestes cenários, o espaço de pesquisa é reduzido e ocorrem poucos conflitos repetidos, limitando o benefício da reutilização de *nogoods*.

À medida que o número máximo de vizinhos aumenta, a topologia torna-se mais densa, originando um crescimento significativo do espaço de pesquisa e do número de conflitos potenciais. Nestes casos, a heurística de **nogood learning demonstra um impacto claro**, reduzindo substancialmente o tempo médio de execução, no qual verificamos:

- Para **5 vizinhos**, a utilização de *nogoods* reduz o tempo médio de execução em cerca de **1 575 ms**, evitando a reexploração de combinações de atribuições já identificadas como inválidas;
- Para **7 vizinhos**, a diferença aumenta para aproximadamente **31 863 ms**, evidenciando que a acumulação e reutilização de *nogoods* se torna cada vez mais relevante à medida que o espaço de pesquisa cresce

Estes resultados confirmam que a eficácia da aprendizagem de conflitos cresce com a complexidade estrutural da topologia, tornando-se particularmente relevante em instâncias mais densas e difíceis.

6.2 Avaliação experimental — impacto do número de sensores (com paralelismo)

Nesta secção analisa-se o impacto do **número de sensores da topologia** no desempenho do algoritmo, considerando **execução paralela ativa**, e comparando os cenários **com aprendizagem de conflitos (nogood learning)** e **sem esta heurística**.

Durante todos os testes apresentados foram mantidos constantes os seguintes parâmetros:

- **Tolerância dos sensores:** 0,5
- **Número máximo de vizinhos por sensor:** 4
- **Execução paralela:** ativa (uso de múltiplas corrotinas)

Assim, o aumento da complexidade resulta exclusivamente do crescimento do número de sensores, permitindo isolar o impacto da dimensão do problema num contexto paralelo. Os testes apresentados nesta secção servem de referência para comparação direta com os resultados **sem paralelismo**, que serão analisados na secção seguinte.

Metodologia experimental

Para cada valor do número de sensores, foi gerada uma topologia aleatória respeitando os parâmetros fixos acima indicados. Cada topologia foi utilizada como entrada do algoritmo em dois cenários distintos:

- com aprendizagem de conflitos (*nogood learning*);
- sem aprendizagem de conflitos.

O tempo médio apresentado corresponde à média aritmética dos tempos obtidos em três execuções independentes do mesmo teste, cada uma com uma topologia aleatória diferente, sendo cada execução composta por cinco forks, perfazendo um total de 15 medições por configuração.

Resultados obtidos

Nº de sensores	WITH NOGOOD — Tempo médio (ms)	WITHOUT NOGOOD — Tempo médio (ms)
10	159,47	153,0
20	1 107,0	1 170,1
30	6 195,4	8 420,7
40	7 862,4	10 191,1
50	26 251,3	35 517,7

Análise dos resultados

Para 10 sensores, a diferença entre as duas versões é residual (aproximadamente **6 ms**), indicando que, para instâncias muito pequenas, a aprendizagem de conflitos não traz benefícios relevantes, mesmo em execução paralela.

A partir de 20 sensores, a versão com *nogood learning* passa consistentemente a apresentar tempos inferiores. Para 30 sensores, a diferença média é de cerca de **2 225 ms**, aumentando para aproximadamente **2 329 ms** aos 40 sensores.

O efeito torna-se particularmente evidente para 50 sensores, onde a utilização de aprendizagem de conflitos reduz o tempo médio de execução em cerca de **9 266 ms** relativamente à versão sem *nogoods*, notando que o crescimento da diferença confirma que, em instâncias maiores, o algoritmo sem aprendizagem de conflitos desperdiça cada vez mais tempo a explorar estados já conhecidos como inválidos.

6.3 Avaliação experimental — impacto do número de sensores (sem paralelismo)

Nesta secção analisa-se o impacto do **número de sensores** no desempenho do algoritmo **em execução sequencial**, isto é, **sem paralelismo**, comparando exclusivamente os cenários **com** e **sem** **aprendizagem de conflitos (nogood learning)**.

Tal como nos testes anteriores, a tolerância dos sensores é mantida em 0,5% e o número máximo de vizinhos por sensor é fixado em 4, garantindo que a variação observada resulta apenas do aumento do tamanho da instância.

Nº de sensores	WITH NOGOOD — Tempo médio (ms)	WITHOUT NOGOOD — Tempo médio (ms)
10	161,9	178,5
20	841,2	1 540,7
30	959,7	1 550,8
40	7 325,8	22 175,6

Análise dos resultados

Para 10 sensores, a diferença entre as duas versões é reduzida (cerca de **16,6 ms**), o que indica que, em instâncias pequenas, o custo adicional de manutenção dos *nogoods* é comparável ao benefício obtido com a poda da pesquisa.

A partir de 20 sensores, o impacto da aprendizagem de conflitos torna-se claro: a versão com *nogoods* executa, em média, **699,5 ms** mais rápido do que a versão sem heurística. Este padrão mantém-se para 30 sensores, com uma diferença semelhante de aproximadamente **591,1 ms**, apesar do aumento do tamanho da instância.

O caso mais expressivo ocorre com 40 sensores, onde a utilização de *nogood learning* reduz o tempo médio de execução em cerca de **14 849,8 ms**. Este resultado evidencia que, em execução sequencial, a ausência de aprendizagem de conflitos conduz rapidamente a uma explosão do espaço de pesquisa, enquanto a versão com *nogoods* consegue reutilizar conflitos previamente identificados para eliminar grandes porções do espaço de procura.

Em síntese, os resultados mostram que, mesmo sem paralelismo, a aprendizagem de conflitos é uma heurística altamente eficaz à medida que o número de sensores aumenta, permitindo controlar o crescimento do tempo de execução e tornando viável a resolução de instâncias de maior dimensão.

6.4 Comparação entre execução paralela e sequencial

Nesta secção analisa-se o impacto do **paralelismo** no desempenho do algoritmo, comparando a execução **paralela** com a execução **sequencial**, mantendo constantes todas as restantes configurações em cada execução (número máximo de 4 vizinhos por sensor e tolerância de 0,5%). A versão paralela recorre a corrotinas de Kotlin executadas sobre o *Dispatchers.Default*, o que utiliza um conjunto de threads gerido automaticamente pela biblioteca para explorar concorrentemente o espaço de pesquisa, em ambos os cenários, a aprendizagem de conflitos (*nogood learning*) encontra-se ativa, garantindo que as diferenças observadas resultam exclusivamente da exploração concorrente do espaço de pesquisa.

Nº de sensores	Com paralelismo — Tempo médio (ms)	Sem paralelismo — Tempo médio (ms)
10	167,33	188,53
20	859,87	1 499,07
30	4 075,1	11 205,9
40	20 238,5	67 331,8

Análise dos resultados

Para 10 sensores, o ganho introduzido pelo paralelismo é modesto, com uma diferença média de aproximadamente **21,2 ms**. Este comportamento é esperado, uma vez que, em instâncias pequenas, o custo de criação, escalonamento e sincronização das corrotinas é comparável ao tempo total de execução, limitando os benefícios da execução concorrente.

A partir de 20 sensores, o impacto do paralelismo torna-se claramente visível, verificando que o tempo médio de execução reduz-se em cerca de **639,2 ms**, refletindo uma exploração mais eficiente do espaço de pesquisa através da distribuição das ramificações por múltiplos trabalhadores.

Com 30 sensores, a diferença aumenta de forma significativa, atingindo aproximadamente **7 130,8 ms**, indicando que o paralelismo permite explorar simultaneamente múltiplas subárvores de pesquisa, reduzindo o tempo total necessário para atingir uma solução ou concluir a enumeração das soluções viáveis.

O caso mais expressivo ocorre com 40 sensores, onde a execução paralela reduz o tempo médio de execução em cerca de **47 093,3 ms**, o que evidencia que, para instâncias de grande dimensão, o paralelismo é essencial para manter tempos de execução aceitáveis, sobretudo quando combinado com aprendizagem de conflitos, que evita a exploração redundante de estados já conhecidos como inválidos.

Com este resultado, é possível demonstrar que o paralelismo tem um impacto crescente à medida que o tamanho da instância aumenta. Em conjunto com a heurística de *nogood learning*, a execução paralela permite escalar o algoritmo para problemas significativamente maiores, reduzindo de forma substancial o tempo total de execução face à versão sequencial.

6.5 Impacto do número de corrotinas na execução paralela

Nesta experiência analisa-se o efeito do **grau de paralelismo**, medido pelo número de corrotinas ativas, no tempo médio de execução do algoritmo, utilizando **5 forks por execução**. Para isolar este fator, foi utilizada **a mesma topologia de rede em todas as medições**, mantendo constantes o número de sensores, o número máximo de vizinhos, a tolerância e a utilização da aprendizagem de conflitos (*nogood learning*).

O único parâmetro variado foi o número máximo de corrotinas permitidas na execução paralela

Nº de corrotinas	Tempo médio (ms)
1	18 932,6
2	10 637,6
3	8 369,6
4	7 376,2
5	6 513,2
6	6 428,6
7	5 622,2
8	5 187,2

Análise dos resultados

Com apenas **1 corrotina**, o algoritmo comporta-se essencialmente como uma execução sequencial, apresentando o maior tempo médio de execução (**18 932,6 ms**). Ao aumentar para **2 corrotinas**, observa-se uma redução substancial de cerca de **8 295,0 ms**, evidenciando o impacto imediato da exploração concorrente do espaço de pesquisa.

Entre **2 e 4 corrotinas**, o ganho continua significativo, embora progressivamente menor. Por exemplo, o aumento de 3 para 4 corrotinas reduz o tempo médio em cerca de **993,4 ms**, refletindo um bom aproveitamento dos recursos de paralelismo disponíveis.

A partir de **5 corrotinas**, os ganhos adicionais tornam-se mais moderados. O aumento de 5 para 6 corrotinas reduz o tempo médio em apenas **84,6 ms**, indicando o início de uma fase de **retornos decrescentes**. Este comportamento é típico em algoritmos paralelos, onde fatores como contenção, sincronização e divisão desigual do espaço de pesquisa limitam a escalabilidade linear.

Apesar disso, o melhor resultado é obtido com **8 corrotinas**, atingindo um tempo médio de **5 187,2 ms**, o que representa uma redução absoluta de **13 745,4 ms** face à execução com uma única corrotina.

Em síntese, os resultados mostram que o aumento do número de corrotinas melhora significativamente o desempenho até um determinado ponto, a partir do qual os ganhos tornam-se marginais, sugerindo que existe um equilíbrio ótimo entre paralelismo e overhead de coordenação, dependente tanto da estrutura da topologia como da complexidade do espaço de pesquisa.

Capítulo 7

Conclusão

Neste trabalho foi desenvolvida e avaliada uma solução para o planeamento automático de *duty cycles* em redes de sensores, assente na modelação da topologia da rede como um grafo e na aplicação de um algoritmo de pesquisa com restrições, permitindo tratar explicitamente os conflitos entre sensores adjacentes, recorrendo à aprendizagem de conflitos (*nogood learning*), e suporta igualmente a obtenção de soluções parciais quando não é possível satisfazer todas as restrições impostas.

O sistema implementado demonstrou a viabilidade da integração entre um algoritmo de planeamento executado no *backend* e uma interface gráfica interativa no *frontend*. Esta integração permite ao utilizador criar, editar e analisar topologias de sensores, executar o algoritmo de escalonamento e visualizar os resultados obtidos, constituindo uma ferramenta funcional para a análise e experimentação de diferentes cenários de rede.

Os resultados experimentais evidenciaram o impacto positivo das técnicas adotadas na eficiência do algoritmo, no qual a utilização de aprendizagem de conflitos revelou-se determinante na redução do tempo de execução em topologias de maior dimensão, enquanto a exploração de paralelismo através de corrotinas permitiu reduzir significativamente o custo computacional. Em particular, observou-se que o aumento do grau de paralelismo conduz a melhorias substanciais no desempenho até um determinado ponto, a partir do qual os ganhos se tornam progressivamente mais reduzidos, refletindo a existência de overheads associados à coordenação concorrente.

De forma global, o trabalho realizado valida a abordagem proposta e confirma a adequação das escolhas de modelação e implementação para o problema em estudo, resultando numa solução funcional, extensível e alinhada com os requisitos do planeamento de *duty cycles* em redes de sensores.

Por fim, este projeto constitui uma base sólida para desenvolvimentos futuros, tanto do ponto de vista algorítmico como ao nível da evolução do sistema, permitindo a incorporação de novas estratégias de otimização e a adaptação a cenários de maior escala e complexidade.

Capítulo 8

Apêndice A

Pseudocódigo do Algoritmo *buildTree*

Este apêndice apresenta o pseudocódigo detalhado do algoritmo *buildTree*, responsável pela exploração da árvore de decisão durante o processo de planeamento automático dos *duty cycles*. O algoritmo recebe como entrada um sensor inicial, os domínios admissíveis, uma atribuição parcial e o contexto de pesquisa, e procede à atribuição recursiva de valores, aplicando heurísticas de ordenação, verificação de restrições e retrocesso (*backtracking*).

Algoritmo *buildTree*(sensor, domains, assignment, currentCost, countSensor, ctx)

1. domain $\leftarrow$ domains[sensor]
2. Se domain = $\emptyset$ então
3. retornar
4. Fim-se
- 5.
6. ordenar domain segundo:
 - (i) menor impacto sobre sensores vizinhos (impactOf)
 - (ii) menor desvio absoluto ao duty-cycle desejado
7. para cada period $\in$ domain fazer
8. percentage $\leftarrow$ [100 / period]
9. assignment[sensor] $\leftarrow$ percentage
10. se assignment viola algum nogood em ctx então
11. remover sensor de assignment
12. continuar
13. fim-se
14. erro $\leftarrow$ percentage – sensor.desiredDutyCycle
15. novoCusto $\leftarrow$ currentCost + erro²
16. rmsParcial $\leftarrow$ $\sqrt{\text{novoCusto} / |\text{assignment}|}$
- 17.
18. se existe bestAssignment
19. e bestNullCount = 0
20. e rmsParcial > bestCost então
21. remover sensor de assignment
22. continuar
23. fim-se
24. emConflito $\leftarrow$ falso
25. para cada vizinho $\in$ vizinhos(sensor) fazer
26. se vizinho $\in$ assignment então
27. se assignment[vizinho] = percentage então
28. emConflito $\leftarrow$ verdadeiro
29. interromper ciclo
30. fim-se

```

29.     se períodosNãoCoprimos(percentage, assignment[vizinho]) então
30.         emConflito ← verdadeiro
31.         interromper ciclo
32.     fim-se
33. fim-se
34. fim-para

35. se emConflito então
36.     conjuntoConflito ← subconjunto de assignment que viola restrições
37.     conjuntoConflito ← conjuntoConflito  $\cup$  { sensor  $\rightarrow$  percentage }
38.     adicionar conjuntoConflito a ctx.localNogoods
39.     adicionar conjuntoConflito a ctx.globalNogoods
40.     remover sensor de assignment
41.     continuar
42. fim-se

43. atualizar melhor solução chamando tryUpdate(assignment, novoCusto) se:
    sensoresEmFalta < bestNullCount
    ou (sensoresEmFalta = bestNullCount e rmsParcial < bestCost)

44. nextSensors ← sensores ainda não atribuídos

45. ordenar nextSensors segundo:
    (i) maior saturação (DSatur)
    (ii) menor domínio (MRV)
    (iii) maior grau na topologia
    (iv) maior risco de não atribuição

46. se nextSensors não vazio então
47.     para cada next  $\in$  nextSensors fazer
48.         buildTree(next, domains, assignment,
                    novoCusto, countSensor + 1, ctx)
49.     fim-para
50. senão se |assignment| < totalSensores então
51.
52.     remover sensor de assignment
53.     retornar
54. fim-se

55.
56. remover sensor de assignment

57. fim-para

```


Apêndice B

Pseudocódigo da Função *impactOf*

Este apêndice apresenta o pseudocódigo da função *impactOf*, utilizada pelo algoritmo de pesquisa para estimar o impacto de uma atribuição de um determinado período a um sensor. Esta função calcula o efeito dessa atribuição sobre os domínios dos sensores vizinhos, permitindo ao algoritmo privilegiar decisões menos restritivas e reduzir o custo da exploração da árvore

Algoritmo: *impactOf*(sensor, period, domains, assignment, ctx, topology)

1: percentage $\leftarrow$ $\lceil 100 / \text{period} \rceil$

2: *impactoTotal* $\leftarrow$ 0

3: Para cada *neighbor* $\in$ *topology.neighbors*(sensor) Faça

4: Se *neighbor* $\in$ *assignment* Então

5: continuar

6: Fim-se

7: *chaveCache* $\leftarrow$ (sensor, period, neighbor)

8: Se *ctx.impactCache* contém *chaveCache* Então

9: *impactoTotal* $\leftarrow$ *impactoTotal* + *ctx.impactCache*[*chaveCache*]

10: continuar

11: Fim-se

12: *domain* $\leftarrow$ *domains*[neighbor]

13: Se *domain* = $\emptyset$ Então continuar

14: *valoresValidos* $\leftarrow$ 0

15: Para cada *valorVizinho* $\in$ *domain* Faça

16: Se *areCoprimePercentages*(
 percentage,
 $\lceil 100 / \text{valorVizinho} \rceil$,
 ctx
) Então

17: *valoresValidos* $\leftarrow$ *valoresValidos* + 1

18: Fim-se

19: Fim-para

20: *delta* $\leftarrow$ |*domain*| – *valoresValidos*

21: *ctx.impactCache*[*chaveCache*] $\leftarrow$ *delta*

22: *impactoTotal* $\leftarrow$ *impactoTotal* + *delta*

23: Fim-para

24: Retornar *impactoTotal*

Referências bibliográficas

- [1] L. Atzori, A. Iera e G. Morabito, *The Internet of Things: A Survey*, **Computer Networks**, vol. 54, n.º 15, pp. 2787–2805, 2010.
- [2] M. M. Alam, O. Berder, D. Menard e O. Sentieys, *Duty-Cycle-Aware MAC Protocols for Wireless Sensor Networks: A Survey*, **IEEE Communications Surveys & Tutorials**, vol. 16, n.º 2, pp. 604–634, 2014.
- [3] D. Passos, C. de Sousa e C. Albuquerque, *Co-Prime Based Asynchronous Duty Cycling: Exact Model and Analysis*, **em submissão**.
- [4] React Documentation, React, 2026. Disponível em: <https://react.dev>. **Último acesso:** 14 de abril de 2026.
- [5] TypeScript Documentation, Microsoft, 2026. Disponível em: <https://www.typescriptlang.org/docs>. **Último acesso:** 14 de abril de 2026.
- [6] Vite Documentation, Vite, 2026. Disponível em: <https://vitejs.dev>. **Último acesso:** 14 de abril de 2026.
- [7] MDN Web Docs – Canvas API, Mozilla, 2026. Disponível em: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API. **Último acesso:** 7 de maio de 2026.
- [8] MDN Web Docs – HTML, CSS e JavaScript, Mozilla, 2026. Disponível em: <https://developer.mozilla.org>. **Último acesso:** 14 de maio de 2026.
- [9] Kotlin Documentation, JetBrains, 2026. Disponível em: <https://kotlinlang.org/docs/home.html>. **Último acesso:** 30 de março de 2026.
- [10] Spring Boot Documentation, VMware, 2026. Disponível em: <https://spring.io/projects/spring-boot>. **Último acesso:** 30 de março de 2026.
- [11] MDN Web Docs – REST, Mozilla, 2026. Disponível em: <https://developer.mozilla.org/en-US/docs/Glossary/REST>. **Último acesso:** 30 de março de 2026.
- [12] PostgreSQL Global Development Group, *PostgreSQL Documentation*, 2026. Disponível em: <https://www.postgresql.org/docs/>. **Último acesso:** 30 de março de 2026.
- [13] R. Dechter, *Constraint Processing*, **1.ª edição**, Morgan Kaufmann, 2003.
- [14] D. Brélaz, "New Methods to Color the Vertices of a Graph", **Communications of the ACM**, vol. 22, n.º 4, pp. 251–256, 1979.